Sapienza Università di Roma
Dipartimento di Fisica

Formulario C++

Sintassi essenziale ed errori da evitare

Caterina Sacchi — A.A. 2026/2027
Versione 1 — Slide 1–244 e integrazioni dagli esami 2022–2025

Come usarlo. Cerca l'operazione che vuoi eseguire e adatta l'esempio. I nomi delle variabili e dei file sono esempi: devono corrispondere a quelli del tuo programma.

1 Compilare ed eseguire

A cosa serve: trasformare il codice sorgente in un eseguibile e avviarlo.

```
clang++ -std=c++17 main.cpp -o programma
./programma
```

- `-std=c++17`: specifica lo standard del linguaggio. Serve almeno C++11 per `auto`, `nullptr`, range-based for e `std::to_string`.
- `main.cpp`: file sorgente da compilare.
- `-o programma`: assegna il nome `programma` all'eseguibile.
- `./programma`: avvia l'eseguibile nella cartella corrente.

Attenzione: salva il sorgente e ricompila dopo ogni modifica. Gli spazi nel comando sono necessari. Se la compilazione fallisce, un vecchio eseguibile può essere ancora presente: avviarlo non verifica il codice appena modificato.

2 Struttura minima di un programma

```
#include <iostream>

int main() {
    std::cout << "Ciao!" << std::endl;
    return 0;
}
```

- `#include <iostream>`: include l'header per input e output standard.
- `int main()`: punto di ingresso del programma.
- Le graffe delimitano il corpo della funzione.
- `return 0;`: termina main con esito positivo.
- Le istruzioni come assegnazioni e stampe terminano con `;`.

Attenzione: `main` deve restituire `int`. La direttiva `#include` non termina con un punto e virgola.

3 Commenti

```
// Commento fino alla fine della riga.

/*
Commento che puo' occupare
piu' righe.
*/
```

A cosa servono: spiegare il codice senza aggiungere istruzioni eseguibili.

4 Variabili

A cosa servono: conservare valori che il programma può leggere e modificare.

```
int numero = 0; // Definizione e inizializzazione.
numero = 123;   // Assegnazione a una variabile esistente.

bool valido = true; // Valore booleano: true oppure false.
```

Sintassi	Significato
`int n;`	Definisce un intero locale senza inizializzarlo.
`int n = 0;`	Definisce un intero e lo inizializza a zero.
`n = 5;`	Assegna un nuovo valore all'intero esistente.
`bool b = false;`	Definisce una variabile booleana inizializzata a falso.

Attenzione: un `int` locale non inizializzato non è automaticamente zero. Assegnagli un valore prima di leggerlo.

5 Output: cout

A cosa serve: scrivere sullo standard output, normalmente il terminale.

```
int numero = 5;

std::cout << "numero"; // Stampa la parola numero.
std::cout << numero;   // Stampa il valore 5.

std::cout << "Valore: " << numero << std::endl;
```

Regola: il testo letterale va tra virgolette; il nome della variabile da leggere va senza virgolette. L'operatore « inserisce dati nel flusso di uscita.

```
std::cout << "A" << "B";    // Stampa AB.
std::cout << "A" << " B";   // Stampa A B.
std::cout << "A\nB\n";     // Stampa su due righe.
std::cout << "A\tB\n";     // Separa con una tabulazione.
```

- `"\n"`: carattere di nuova riga.
- `"\t"`: tabulazione orizzontale.
- `std::endl`: nuova riga e svuotamento del buffer di uscita.

Attenzione: `cout` non aggiunge automaticamente spazi. Andare a capo nel sorgente non manda a capo l'output.

6 Input: cin

A cosa serve: leggere dallo standard input, normalmente la tastiera.

```cpp
int numero = 0;

std::cout << "Inserisci un intero: ";
std::cin >> numero;
```

L'operatore » estrae un valore dal flusso e lo memorizza nella variabile. Il tipo della variabile determina il tipo di lettura.

Controllare una lettura fallita

```cpp
if (std::cin.fail()) {
    std::cout << "Errore nella lettura!" << std::endl;
    return 1;
}
```

`fail()` controlla lo stato di errore del flusso. In `main`, `return 1;` termina il programma segnalando un esito non positivo.

Attenzione: questo controllo non verifica che tutta la riga contenga esclusivamente un intero. Con input `3.5`, la lettura in un `int` acquisisce `3` e lascia `.5` nel flusso: la lettura può quindi riuscire.

7 Namespace: std::

A cosa serve: specificare a quale spazio dei nomi appartiene un nome.

```cpp
std::cout << "Ciao!" << std::endl;
```

Negli esempi del professore compare spesso questa alternativa:

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "Ciao!" << endl;
    return 0;
}
```

`using namespace std;` permette di cercare i nomi non qualificati anche in `std`. Non include una libreria e non sostituisce la direttiva `#include`.

8 Condizioni: if

A cosa serve: eseguire un'istruzione o un blocco quando una condizione è vera.

```cpp
if (numero == 5) {
    std::cout << "Il numero vale 5" << std::endl;
}
```

Per una sola istruzione, le graffe possono essere omesse:

```cpp
if (numero == 5) std::cout << "Vale 5" << std::endl;
```

Attenzione: senza graffe, l'if controlla soltanto l'istruzione immediatamente successiva. Non mettere un punto e virgola subito dopo `if (condizione)`.

Operatori incontrati

Operatore	Significato	Esempio
`==`	Uguale a	`numero == 5`
`!=`	Diverso da	`numero != 0`
`<`	Minore di	`numero < 10`
`>=`	Maggiore o uguale a	`numero >= 0`
`!`	Negazione logica	`!valido`

```
bool truth = (1 != 0); // Il confronto produce true.

if (truth) {
    std::cout << "Condizione vera" << std::endl;
}
```

Da ricordare: `=` assegna un valore; `==` confronta due valori. La negazione `!` trasforma `true` in `false` e viceversa.

9 Ripetizioni: for

A cosa serve: ripetere un blocco controllando un contatore.

```
for (int i = 0; i < 5; ++i) {
    std::cout << i << std::endl;
}
// Stampa 0, 1, 2, 3, 4 su righe separate.
```

Parte	Quando viene eseguita
`int i = 0`	Una volta, all'inizio: crea e inizializza il contatore.
`i < 5`	Prima di ogni giro: se è falsa, il ciclo termina.
`++i`	Dopo ogni giro: aumenta il contatore di uno.

Passo diverso da uno

```
int limite = 7;
int step = 3;

for (int index = 0; index < limite; index += step) {
    std::cout << index << std::endl;
}
// Stampa 0, 3, 6.
```

In questo esempio:

```
index += step; // Equivale a index = index + step.
++index;       // Aumenta index di 1.
```

Attenzione: il limite escluso non coincide necessariamente con il numero di ripetizioni. Con limite `7` e passo `3` si eseguono tre ripetizioni.

Con limite positivo e passo zero, questo ciclo non termina. Per gli esempi usiamo passi positivi e valori abbastanza piccoli da non superare l'intervallo rappresentabile dal contatore.

La variabile dichiarata nell'intestazione del `for` è utilizzabile nel ciclo, ma non dopo la sua chiusura.

10 Array: conservare più valori

Slide 33–34.

Un array contiene un numero fisso di elementi dello stesso tipo, memorizzati consecutivamente.

```
float v1[3] = {0.4f, 1.34f, 56.156f};
float v2[3] = {}; // Tutti gli elementi inizializzati a zero.
int v3[] = {1, 2, 3, 4, 5, 6, 7}; // Dimensione dedotta: 7.
```

```
std::cout << v1[0]; // Primo elemento.
v1[2] = 10.0f;      // Modifica il terzo elemento.
```

Il suffisso `f` indica un letterale di tipo `float`; senza suffisso, un letterale decimale come `0.4` ha tipo `double`.

Indici validi: per un array di dimensione N, da 0 a $N - 1$.

```
for (int i = 0; i < 3; ++i) {
    std::cout << v1[i] << std::endl;
}
```

Attenzione alle slide: `float v2[3];` dentro una funzione non inizializza gli elementi a zero. Leggere elementi non inizializzati o accedere fuori dai limiti produce comportamento indefinito: non esiste un output su cui fare affidamento.

11 Tipi e sizeof

Slide 36–37.

Tipo	Uso
`bool`	Valori logici: true e false.
`char`	Un carattere, ad esempio `'c'`.
`short, int, long, long long`	Numeri interi.
`float`	Numeri in virgola mobile.
`double`	Virgola mobile, almeno la precisione di float.
`long double`	Almeno la precisione di double.

```
char lettera = 'c';
double misura = 1.243e-4; // 1.243 * 10^(-4).

std::cout << sizeof(int) << std::endl;
std::cout << sizeof(misura) << std::endl;
```

`sizeof` restituisce la dimensione in byte. Il risultato ha tipo `std::size_t`. Le dimensioni dipendono dall'implementazione: non memorizzare la tabella della slide come universale.

Vale:

$$\text{sizeof}(\texttt{char}) = 1,$$

$$\text{sizeof}(\texttt{short}) \leq \text{sizeof}(\texttt{int}) \leq \text{sizeof}(\texttt{long}) \leq \text{sizeof}(\texttt{long long}).$$

12 Funzioni: separare e riutilizzare operazioni

Slide 38–39.

```
// Definizione: tipo restituito, nome, parametri e corpo.
double times_pi(double a) {
    return 3.14 * a;
}

// void indica che non viene restituito un valore.
void question() {
    std::cout << "Inserisci un numero: " << std::endl;
}
```

Dentro `main`, si chiamano così:

```
question();
double risultato = times_pi(2.0);
```

Parametro: `a`, nella definizione. **Argomento:** `2.0`, nella chiamata. `return` termina la funzione e, se previsto, restituisce il risultato.

Dichiarare prima di chiamare

Se la definizione si trova dopo la chiamata, serve prima una dichiarazione:

```
void question(); // Dichiarazione: termina con ;.

// Qui puo' comparire main, che chiama question().

void question() { // Definizione.
    std::cout << "Inserisci un numero: " << std::endl;
}
```

Per questa dichiarazione di funzione, `extern` usato nella slide non è necessario. Una definizione può anche trovarsi in un altro file: il linker collega la chiamata alla definizione.

13 Scope: dove un nome è utilizzabile

Slide 40.

Le graffe introducono blocchi. Una variabile locale è utilizzabile nel proprio ambito, a partire dalla dichiarazione.

```
int x = 1;

{
    int y = 2;
    x = 3; // La variabile esterna e' accessibile.
}
// y non e' accessibile qui; x vale 3.
```

Shadowing: un nome nasconde un altro nome

```
int x = 1;
{
    int x = 4; // Una nuova variabile distinta.
    std::cout << x; // Stampa 4.
}
std::cout << x; // Stampa 1.
```

Attenzione: definire una nuova variabile con lo stesso nome non modifica quella esterna. Usare nomi distinti rende il codice più chiaro.

14 Definire namespace

Slide 41–45.

Un namespace raggruppa dichiarazioni e permette di distinguere nomi uguali.

```
namespace physics {
    double mean(double a, double b) {
        return (a + b) / 2.0;
    }
}
```

Dentro una funzione:

```
double m = physics::mean(3.0, 4.0); // Vale 3.5.
```

`::` qualifica il nome. Importare con `using namespace` più namespace che contengono funzioni omonime può rendere una chiamata ambigua.

Una direttiva `using namespace std;` scritta dentro una funzione vale in quell'ambito: non rende `cout` disponibile senza `std::` in tutte le altre funzioni.

15 Puntatori e riferimenti

Slide 46–49.

Un **puntatore** contiene un indirizzo. Un **riferimento** è un altro nome per un oggetto esistente.

```
int a = 23;
int* b = &a; // Puntatore all'oggetto a.
int& x = a;  // Riferimento all'oggetto a.

*b = 3; // Modifica a: ora vale 3.
x = 45; // Modifica a: ora vale 45.
```

Espressione	Significato
`a`	Valore dell'intero.
`&a`	Indirizzo dell'intero.
`b`	Indirizzo conservato nel puntatore.
`*b`	Oggetto a cui il puntatore punta.
`&b`	Indirizzo del puntatore stesso.
`x`	Oggetto a, tramite il riferimento.

Il significato di `*` e `&` dipende dal contesto: in una dichiarazione indicano tipi puntatore e riferimento; in un'espressione indicano dereferenziazione e ottenimento dell'indirizzo.

Cambiare il puntatore o il dato

```
int a = 1, c = 3;
int* p = &a;

*p = 5; // Modifica a.
p = &c; // Cambia l'oggetto a cui p punta.
```

Un riferimento deve essere inizializzato e non può essere ricollegato a un altro oggetto:

```
int& r = a;
r = c; // Copia il valore di c in a.
```

Puntatori non validi

```
int* p = nullptr; // Non punta a un oggetto.
// *p = 3;       // ERRORE: dereferenziazione non valida.
```

Attenzione: inizializza i puntatori prima di usarli. Dereferenziare un puntatore nullo, fuori dai limiti o relativo a un oggetto che non esiste più produce comportamento indefinito. Un programma che compila può comunque essere errato.

Gli indirizzi stampati possono cambiare tra esecuzioni: non sono valori da memorizzare.

16 Parametri: valore, riferimento e puntatore

Slide 50–51.

```
void perValore(double x) {
    x = 1.234; // Modifica soltanto la copia locale.
}

void perRiferimento(double& x) {
    x = 1.234; // Modifica l'oggetto del chiamante.
}

void perPuntatore(double* x) {
    *x = 1.234; // Richiede un puntatore valido.
}
```

Dentro `main`:

```
double a = 5.0;

perValore(a);      // a rimane 5.0.
perRiferimento(a); // a diventa 1.234.
perPuntatore(&a);  // Passa l'indirizzo di a.
```

Regola: per valore si passa una copia. Per riferimento si accede all'oggetto originale. Un parametro puntatore riceve una copia dell'indirizzo: tramite la dereferenziazione può modificare l'oggetto.

Precisazione sulla slide 48: in una funzione `print_info(int var, ...)`, `&var` è l'indirizzo della copia locale, non quello della variabile del chiamante.

17 Array e aritmetica dei puntatori

Slide 34 e 52–53.

In molte espressioni il nome di un array viene convertito in un puntatore al primo elemento. L'array e il puntatore rimangono tipi distinti.

```
int v[] = {10, 20, 300, 40, 50, 60, 70};
int* p = v; // Equivale qui a &v[0].

std::cout << *p;       // 10.
std::cout << *(p + 2); // 300.
std::cout << v[2];     // 300.

++p; // Ora punta a v[1].
```

`p + k` avanza di `k` elementi, non di `k` byte. La distanza in byte dipende da `sizeof` del tipo degli elementi.

```
int* primo = &v[0];
int* quinto = &v[4];

auto distanza = quinto - primo; // 4 elementi.
```

La differenza tra puntatori allo stesso array ha tipo `std::ptrdiff_t`.

Limiti: si possono formare puntatori agli elementi o alla posizione immediatamente successiva all'ultimo elemento. Quest'ultima posizione non può essere dereferenziata. Formare puntatori ancora oltre quei limiti è già comportamento indefinito.

Preincremento e postincremento

```
int i = 0;
int a = i++; // a vale 0; poi i diventa 1.
int b = ++i; // i diventa 2; b vale 2.
```

La stessa distinzione vale per i puntatori: `p++` produce il vecchio puntatore e poi lo avanza; `++p` lo avanza e produce quello aggiornato.

Nomi costruiti per la stampa

```
#include <string>

// Dentro una funzione:
std::string nome = "v[" + std::to_string(2) + "]";
```

`std::string` rappresenta una stringa; `std::to_string` converte un numero in testo. Qui `nome` contiene `"v[2]"`.

18 const: limitare le modifiche

Slide 54–56.

```
const double pi = 3.141592653589793;
// pi = 3; // Non consentito.
```

Una costante può essere inizializzata anche con un valore calcolato durante l'esecuzione: `const` non significa necessariamente "noto in compilazione".

Negli esempi seguenti, `a` e `c` sono gli interi definiti nel primo esempio qui sotto.

Puntatore a dato costante

```
int a = 1, c = 3;
const int* p1 = &a;

// *p1 = 5; // Non consentito tramite p1.
p1 = &c;    // Consentito.
a = 5;      // Consentito: a non e' const.
```

Puntatore costante

```
int* const p2 = &a;

*p2 = 5;    // Consentito.
// p2 = &c; // Non consentito.
```

Puntatore costante a dato costante

```
const int* const p3 = &a;

// Non si puo' cambiare p3.
// Non si puo' modificare il dato tramite p3.
```

Come leggerli: il `const` a sinistra di `*` protegge il dato; il `const` a destra di `*` protegge il puntatore.

Riferimento a dato costante

```
void mostra(const double& x) {
    std::cout << x;
    // x = 1.234; // Non consentito.
}
```

Uso: accedere a un oggetto senza copiarlo e senza modificarlo tramite quel parametro.

19 Applicazione: media

Slide 58–60.

Per N dati:

$$\bar{x} = \frac{1}{N} \sum_{i=0}^{N-1} x_i.$$

```
double mean(const double* data, int nData) {
    double somma = 0.0;

    for (int i = 0; i < nData; ++i) {
        somma += data[i];
    }

    return somma / nData;
}
```

Dentro `main`:

```
double pressure[] = {1.2, 0.9, 1.34, 1.67,
                     0.87, 1.04, 0.76};

double average = mean(pressure, 7);
```

Condizioni d'uso: `nData > 0` e almeno `nData` elementi validi disponibili a partire da `data`.

`const double* data` impedisce alla funzione di modificare gli elementi tramite questo puntatore. La funzione restituisce il risultato con `return`.

Nelle slide compare anche un risultato passato per riferimento:

```
void computeMean(const double* data,
                 int nData, double& risultato) {
    risultato = mean(data, nData);
}
```

Per un solo risultato, restituire direttamente il valore rende spesso più chiara la chiamata. Una funzione `void` ha comunque un tipo di ritorno: `void` indica l'assenza di un valore restituito.

Errore della slide 60: `mean(pressure, 8)` è una chiamata non valida per l'array di sette elementi. Il puntatore non comunica automaticamente alla funzione la dimensione dell'array.

20 Applicazione: deviazione standard

Slide 61: implementazione dell'esercizio.

Occorre scegliere quale definizione usare. La deviazione standard con denominatore N è:

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=0}^{N-1}(x_i - \bar{x})^2}.$$

```
#include <cmath> // Per std::sqrt.

// Richiede la funzione mean definita in precedenza.
double stdDev(const double* data, int nData) {
    double media = mean(data, nData);
    double sommaQuadrati = 0.0;

    for (int i = 0; i < nData; ++i) {
        double differenza = data[i] - media;
        sommaQuadrati += differenza * differenza;
    }

    return std::sqrt(sommaQuadrati / nData);
}
```

Valgono le stesse condizioni d'uso di `mean`.

Se si vuole la deviazione standard campionaria basata sulla varianza corretta:

$$s = \sqrt{\frac{1}{N-1}\sum_{i=0}^{N-1}(x_i - \bar{x})^2},$$

si divide per `nData - 1` e si richiede `nData > 1`. La scelta dipende dalla richiesta dell'esercizio.

21 Come scegliere il parametro di una funzione

Parametro	Effetto
`T x`	Copia il valore; modificare x non modifica l'originale.
`T& x`	Accede all'originale e può modificarlo.
`const T& x`	Accede all'originale senza modificarlo tramite x.
`T* p`	Riceve un indirizzo; può modificare il dato tramite *p.
`const T* p`	Riceve un indirizzo; non può modificare il dato tramite *p.

Qui `T` rappresenta un tipo generico, ad esempio `int` o `double`.

Controllo finale: compilazione riuscita, esecuzione riuscita e risultato corretto sono tre verifiche distinte.

22 Opzioni del compilatore

Slide 63–65.

Le slide usano `g++`; sul nostro Mac utilizziamo `clang++`. Le opzioni riportate qui sono utilizzabili con entrambi.

Opzione	Scopo
`-std=c++17`	Specifica lo standard del linguaggio (almeno `c++11` per gli esempi di questo formulario).
`-o nome`	Specifica il nome del file prodotto.
`-E`	Esegue soltanto il preprocessamento.
`-c`	Produce un file oggetto senza eseguire il linking.
`-Ipercorso`	Aggiunge una cartella in cui cercare header.
`-Lpercorso`	Aggiunge una cartella in cui cercare librerie.
`-lnome`	Richiede il collegamento alla libreria indicata.
`-Wall`	Abilita un insieme di avvisi comuni.
`-Werror`	Tratta gli avvisi come errori.
`-g`	Produce informazioni per il debugger.

```
# Compilazione per studiare e trovare errori.
clang++ -std=c++17 -Wall -g main.cpp -o programma

# Avvisi trattati come errori.
clang++ -std=c++17 -Wall -Werror main.cpp -o programma

# Produce main.o, senza creare un eseguibile.
clang++ -std=c++17 -c main.cpp -o main.o

# Collega il file oggetto e crea l'eseguibile.
clang++ main.o -o programma

# Compila e collega piu' sorgenti.
clang++ -std=c++17 main.cpp funzioni.cpp -o programma
```

Attenzione: `-Wall` non abilita letteralmente tutti gli avvisi. Un warning può segnalare un errore serio anche quando la compilazione riesce.

```
int index() {
    int i = 27;
    return i; // Necessario per restituire il valore.
}
```

Arrivare alla fine di una funzione che deve restituire un valore, senza restituirlo, produce comportamento indefinito. `main` è un caso speciale: raggiungerne la fine equivale a `return 0;`.

Le istruzioni poste dopo un `return`, nello stesso percorso di esecuzione, non vengono eseguite.

23 Simboli e file oggetto: nm

Slide 66.

```
nm -C main.o
```

`nm` elenca i simboli presenti nel file. L'opzione `-C` rende leggibili i nomi C++ codificati internamente: questa operazione si chiama *demangling*.

Un simbolo indicato con `U` è referenziato ma non definito in quel file. In un file oggetto può essere normale: la definizione deve essere risolta durante il linking.

24 Ottimizzazione

Slide 67–68.

Opzione	Uso generale
`-O0`	Disabilita la maggior parte delle ottimizzazioni; utile per seguire il codice nel debugger.
`-O1`	Abilita un primo livello di ottimizzazione.
`-O2`	Abilita ulteriori ottimizzazioni.
`-O3`	Abilita ottimizzazioni più aggressive; non garantisce di essere sempre più veloce.
`-Os`	Privilegia la riduzione della dimensione.
`-Ofast`	Può consentire trasformazioni che non rispettano tutte le regole standard, soprattutto per i calcoli in virgola mobile.

```
# Sviluppo e debugging.
clang++ -std=c++17 -Wall -g -O0 main.cpp -o programma

# Eseguibile ottimizzato.
clang++ -std=c++17 -Wall -O2 main.cpp -o programma
```

Attenzione: l'ottimizzazione non corregge un programma errato. In presenza di comportamento indefinito, il risultato può cambiare anche drasticamente.

Per calcoli scientifici, non adottare `-Ofast` senza valutarne gli effetti numerici. Le opzioni precise e la loro disponibilità dipendono dal compilatore e dalla versione.

25 Debugging con LLDB

Slide 69–73.

Un debugger permette di fermare l'esecuzione, ispezionare variabili e seguire le istruzioni.

Prima compila con informazioni di debugging:

```
clang++ -std=c++17 -Wall -g -O0 main.cpp -o programma
lldb ./programma
```

I comandi seguenti si scrivono dentro LLDB, dopo il prompt `(lldb)`:

```
breakpoint set --name main
run
next
print numero
bt
continue
quit
```

Comando	Scopo
`run`	Avvia il programma.
`breakpoint set -name main`	Imposta un punto di arresto all'ingresso di main.
`breakpoint set -file main.cpp -line 20`	Imposta un punto di arresto sulla riga indicata.
`next`	Avanza senza entrare nelle funzioni chiamate.
`step`	Avanza entrando nelle funzioni chiamate.
`finish`	Prosegue fino all'uscita dalla funzione corrente.
`print numero`	Valuta e mostra un'espressione.
`bt`	Mostra la sequenza delle chiamate attive.
`continue`	Riprende l'esecuzione.
`quit`	Esce dal debugger.

Puntatore e dato sono distinti: `print p` mostra il puntatore; `print *p` tenta di accedere all'oggetto puntato. Se il puntatore non è valido, quell'accesso non è lecito.

Precisazione: `-g` aggiunge informazioni di debugging, ma da solo non implica necessariamente un rallentamento dell'esecuzione. L'ottimizzazione può rendere più difficile seguire le variabili e le righe del sorgente.

26 Analisi statica: cppcheck

Slide 74.

```
cppcheck main.cpp
```

Se installato, `cppcheck` analizza il sorgente senza eseguire il programma. Può individuare problemi come variabili non inizializzate e dereferenziazioni di puntatori nulli.

Attenzione: nessun singolo strumento garantisce di trovare tutti gli errori. Analisi statica, avvisi del compilatore, debugging e verifica dei risultati si completano.

27 Classi e oggetti

Slide 76–81.

Una **classe** definisce un nuovo tipo, raggruppando dati e operazioni. Un **oggetto** è un'istanza di quella classe.

- **Attributi o data members:** dati conservati nell'oggetto.
- **Metodi o member functions:** funzioni appartenenti alla classe.
- **Interfaccia pubblica:** operazioni accessibili a chi usa la classe.

```
class Result {
public:
    Result() {
        mean_ = 0.0;
        stdDev_ = 0.0;
    }

    Result(double mean, double stdDev) {
        mean_ = mean;
        stdDev_ = stdDev;
    }

    // const: vedi la sezione "Metodi const".
    double getMean() const { return mean_; }
    double getStdDev() const { return stdDev_; }

private:
    double mean_;
    double stdDev_;
}; // Punto e virgola obbligatorio.
```

Dentro `main`:

```
Result r1;             // Costruttore senza argomenti.
Result r2(1.1, 0.234);  // Costruttore con argomenti.

std::cout << r2.getMean(); // Accesso tramite il punto.
```

Ogni oggetto conserva i propri attributi: modificare un oggetto non modifica automaticamente gli altri.

28 public, private e incapsulamento

Slide 82–86.

- `public:` accessibile dall'esterno.
- `private:` accessibile ai membri della classe e agli eventuali friend autorizzati.

In una `class`, i membri sono `private` per default.

```
Result r(1.1, 0.234);

double m = r.getMean(); // Consentito.
// double m = r.mean_; // Non consentito: membro privato.
```

Incapsulamento: chi usa la classe si affida all'interfaccia pubblica, mentre i dettagli interni rimangono nascosti. Si possono cambiare nomi e organizzazione degli attributi senza modificare il codice dei clienti, se l'interfaccia e il suo comportamento restano compatibili.

Anche un metodo può essere privato:

```
class Datum {
public:
    Datum() { reset(); }

private:
    void reset() {
        value_ = 0.0;
        error_ = 0.0;
    }

    double value_;
    double error_;
};
```

Il costruttore può chiamare `reset()`. Il codice esterno non può chiamare `d.reset()`.

Precisazione sulla slide 81: una direttiva `using namespace std;` non è ammessa direttamente nel corpo di una classe. Negli esempi usiamo nomi qualificati, come `std::string` e `std::cout`.

29 Costruttori

Slide 87–89.

Un costruttore inizializza un nuovo oggetto: ha lo stesso nome della classe e non ha tipo di ritorno, nemmeno `void`.

```
class Datum {
public:
    // Costruttore di default.
    Datum() {
        value_ = 0.0;
        error_ = 0.0;
    }

    // Costruttore con due parametri.
    Datum(double value, double error) {
        value_ = value;
        error_ = error;
    }

    // Costruttore di copia.
    Datum(const Datum& other) {
        value_ = other.value_;
        error_ = other.error_;
    }

private:
    double value_;
    double error_;
};
```

```
Datum d1;             // Default.
Datum d2(0.23, 0.212); // Con parametri.
Datum d3(d2);         // Copia: crea un nuovo oggetto.
```

Attenzione: un costruttore vuoto come `Datum() {}` non inizializza automaticamente a zero i membri numerici degli esempi. Lo zero mostrato nelle slide non è garantito.

Per classi semplici come questa, il costruttore di copia generato dal compilatore copia già i singoli membri: quello esplicito serve qui a mostrare il meccanismo.

Distinzione: `Datum d3(d2);` costruisce un nuovo oggetto; `d3 = d2;` assegna a un oggetto già esistente.

30 Getter, setter e metodi di utilità

Slide 90–92.

```
#include <iostream>

class Datum {
public:
    Datum(double value, double error) {
        value_ = value;
        error_ = error;
    }

    // Getter: leggono gli attributi (const).
    double value() const { return value_; }
    double error() const { return error_; }

    // Setter: modificano gli attributi (non const).
    void setValue(double value) { value_ = value; }
    void setError(double error) { error_ = error; }

    // Metodo di utilita': stampa l'oggetto (const).
    void print() const {
        std::cout << value_ << " +/- " << error_
                  << std::endl;
    }

private:
    double value_;
    double error_;
};
```

```
Datum d(23.4, 7.5);

double x = d.value();
d.setValue(8.563);
d.setError(0.45);
d.print();
```

Perché const nei getter e in print: il `const` dopo le parentesi indica che il metodo non modifica l'oggetto (vedi la sezione *Metodi const*). Senza di esso, questi metodi non si possono chiamare su oggetti const né tramite riferimenti const, ad esempio nel ciclo `for (const Datum& d : data)`.

Attenzione: un setter dovrebbe rispettare i vincoli del dato. Questo esempio assegna direttamente i valori e non verifica, ad esempio, che l'incertezza sia non negativa.

Il suffisso `_` nei nomi degli attributi è una convenzione, non una regola del linguaggio.

31 Overloading: stesso nome, parametri diversi

Esempio Counter, slide 93.

```
void increment() {
    count_++;
}

void increment(int step) {
    count_ += step;
}
```

Questi metodi possono appartenere alla stessa classe. La chiamata seleziona la versione in base agli argomenti:

```
counter.increment();  // Incrementa di 1.
counter.increment(7); // Incrementa di 7.
```

Anche i costruttori possono essere sovraccaricati. Cambiare soltanto il tipo di ritorno non basta a distinguere due funzioni.

32 Puntatori e riferimenti a oggetti

Slide 93–94.

```
Datum d(1.2, 0.1);
Datum* p = &d;
Datum& r = d;

d.print();  // Oggetto: usa il punto.
r.print();  // Riferimento: usa il punto.
p->print(); // Puntatore: usa la freccia.

(*p).print(); // Equivale a p->print().
```

Situazione	Accesso a un metodo
Oggetto	`oggetto.metodo()`
Riferimento	`riferimento.metodo()`
Puntatore	`puntatore->metodo()`

Il puntatore deve riferirsi a un oggetto valido.

Passare oggetti alle funzioni

```
void printByRef(Datum& d) {
    d.print();
}

void printByPtr(Datum* d) {
    d->print(); // Richiede un puntatore valido.
}
```

```
printByRef(d);
printByPtr(&d);
```

Il passaggio per riferimento o puntatore permette di accedere all'oggetto originale. Il passaggio per valore crea invece un parametro distinto, inizializzato dall'argomento.

33 this e dimensione degli oggetti

Slide 93–94.

Dentro un metodo non statico, `this` è il puntatore all'oggetto sul quale il metodo è stato chiamato.

Ad esempio, nel corpo di un metodo:

```
std::cout << "Indirizzo dell'oggetto: "
          << this << std::endl;
```

```
Datum d(1.2, 0.1);
Datum* p = &d;

std::cout << sizeof(d) << std::endl;
std::cout << sizeof(p) << std::endl;
```

- `sizeof(d)` misura lo spazio occupato dall'oggetto.

- `sizeof(p)` misura lo spazio occupato dal puntatore, non dall'oggetto puntato.

La dimensione dell'oggetto può includere spazi di allineamento, detti *padding*. Non è sempre la semplice somma delle dimensioni degli attributi.

Precisazioni sulle slide: la dimensione di un puntatore non coincide universalmente con quella di `long`. La coincidenza tra indirizzo dell'oggetto e indirizzo del primo attributo non è una regola da applicare a qualsiasi classe: dipende dalle proprietà del suo layout.

34 Applicazione: restituire un risultato composto

Collegamento con la slide 76.

Una classe permette di restituire insieme quantità logicamente collegate, come media e deviazione standard.

Assumendo già definite `Result`, `mean` e `stdDev`:

```
Result mean_and_stdev(const double* data, int nData) {
    double m = mean(data, nData);
    double s = stdDev(data, nData);

    return Result(m, s);
}
```

Dentro `main`:

```
double dati[] = {1.2, 0.9, 1.34};

Result risultato = mean_and_stdev(dati, 3);

std::cout << risultato.getMean() << " +/- "
          << risultato.getStdDev() << std::endl;
```

Questa è un'applicazione costruita a partire dall'idea della slide. Valgono le condizioni d'uso delle funzioni statistiche: numero di dati positivo e accessi entro i limiti.

Schema da ricordare: la classe definisce il tipo; il costruttore inizializza l'oggetto; gli attributi conservano i dati; i metodi forniscono le operazioni; `public` definisce l'interfaccia accessibile.

35 Separare header, implementazione e applicazione

Slide 96–107.

Per organizzare un programma su più file:

File	Contenuto
`Datum.h`	Definizione della classe e dichiarazioni dei suoi metodi.
`Datum.cc`	Definizioni dei metodi.
`main.cpp`	Applicazione che usa la classe.
`Datum.o`	File oggetto prodotto compilando Datum.cc.

Le estensioni `.cc` e `.cpp` sono entrambe usate per sorgenti C++.

Header: Datum.h

```
#ifndef DATUM_H
#define DATUM_H

class Datum {
public:
    Datum(double value, double error);

    double value() const;
    double error() const;
```

```
    void setValue(double value);
    void setError(double error);

    void print() const;

private:
    double value_;
    double error_;
};

#endif
```

Implementazione: Datum.cc

```
#include "Datum.h"
#include <iostream>

Datum::Datum(double value, double error) {
    value_ = value;
    error_ = error;
}

double Datum::value() const {
    return value_;
}

double Datum::error() const {
    return error_;
}

void Datum::setValue(double value) {
    value_ = value;
}

void Datum::setError(double error) {
    error_ = error;
}

void Datum::print() const {
    std::cout << value_ << " +/- " << error_
              << std::endl;
}
```

`Datum::` specifica che il metodo appartiene alla classe Datum. Le definizioni devono corrispondere alle dichiarazioni, anche per la presenza di `const`.

Applicazione: main.cpp

```
#include "Datum.h"

int main() {
    Datum d(23.4, 7.5);
    d.print();

    d.setValue(8.563);
    d.setError(0.45);
    d.print();

    return 0;
}
```

Compilazione

```
# Compila l'implementazione senza linking.
clang++ -std=c++17 -Wall -c Datum.cc -o Datum.o

# Compila l'applicazione e collega il file oggetto.
clang++ -std=c++17 -Wall main.cpp Datum.o -o programma

./programma
```

Alternativa in un solo comando:

```
clang++ -std=c++17 -Wall main.cpp Datum.cc -o programma
```

Attenzione: includi `Datum.h`, non `Datum.cc`. L'inclusione inserisce testo nel sorgente; il linking collega i file oggetto compilati.

Se modifichi `Datum.cc`, ricompila `Datum.o` e ricollega l'applicazione. Se modifichi l'header, ricompila tutti i sorgenti che ne dipendono.

36 Include guards

Slide 108.

Le protezioni dell'header impediscono di ripeterne il contenuto quando viene incluso più volte nella stessa unità di traduzione.

```
#ifndef DATUM_H
#define DATUM_H

// Contenuto dell'header.

#endif
```

- `#ifndef`: controlla che la macro non sia già definita.
- `#define`: definisce la macro.
- `#endif`: chiude il blocco condizionale.

`DATUM_H` è una macro del preprocessore, non una variabile C++. Usa nomi distinti per proteggere header diversi.

Gli header devono includere gli header necessari ai tipi che utilizzano: ad esempio `<string>` se dichiarano parametri `std::string`.

37 Metodi definiti nell'header: inline

Slide 101.

Un metodo definito direttamente dentro la classe è implicitamente inline nel normale uso degli header:

```
class Datum {
public:
    double value() const { return value_; }

private:
    double value_;
};
```

Significato: inline permette, nel rispetto delle regole del linguaggio, definizioni identiche in più unità di traduzione. Non garantisce che il compilatore sostituisca la chiamata con il corpo del metodo.

Per metodi brevi, come i getter, la definizione nell'header può essere comoda.

38 Classi nei namespace

Slide 109–110.

```
namespace rome {
namespace teaching {

class Datum {
public:
    Datum(double value, double error);

private:
    double value_;
    double error_;
};

}
}
```

Definizione del costruttore nel sorgente:

```
rome::teaching::Datum::Datum(double value, double error) {
    value_ = value;
    error_ = error;
}
```

Uso dentro una funzione:

```
rome::teaching::Datum d(23.4, 7.5);
```

In alternativa si possono scrivere le definizioni all'interno degli stessi blocchi namespace. Il nome qualificato distingue la classe da eventuali classi omonime.

39 std::vector: sequenze a dimensione variabile

Slide 112–113.

```
#include <vector>

// Dentro una funzione:
std::vector<double> valori;

valori.push_back(1.3);
valori.push_back(-2.1);
```

`std::vector<T>` conserva elementi di tipo `T` e gestisce automaticamente la memoria.

Operazione	Scopo
`v.push_back(x)`	Aggiunge un elemento in fondo.
`v.size()`	Restituisce il numero di elementi.
`v[i]`	Accede all'elemento di indice i, senza controllo dei limiti.
`v.at(i)`	Accede all'elemento, controllando i limiti.
`v.begin()`	Iteratore al primo elemento.
`v.end()`	Iteratore alla posizione dopo l'ultimo.

Attenzione: gli indici validi sono da zero a `v.size()-1`, se il vector non è vuoto. `at()` segnala un indice non valido con un'eccezione; `[]` non lo controlla.

Aggiungere elementi può riallocare la memoria e invalidare puntatori, riferimenti e iteratori agli elementi precedenti.

Vector di oggetti

```
std::vector<Datum> data;

data.push_back(Datum(1.3, 0.2));
data.push_back(Datum(-2.1, 0.3));

data[0].print();
```

Qui `data[0]` è un oggetto Datum: per chiamarne i metodi si usa il punto.

40 Tre modi di attraversare un vector

Slide 113.

Con gli indici

```
for (std::size_t i = 0; i < data.size(); ++i) {
    data[i].print();
}
```

`size()` restituisce un tipo intero senza segno. Usare `std::size_t` per l'indice evita il confronto tra un int con segno e una dimensione senza segno.

Con gli iteratori

```
for (std::vector<Datum>::iterator it = data.begin();
     it != data.end(); ++it) {
    it->print();
}
```

Un iteratore permette di individuare e attraversare gli elementi del contenitore. `end()` non indica un elemento leggibile: è il limite escluso del ciclo.

Con il range-based for

```
for (const Datum& d : data) {
    d.print(); // Richiede print() const.
}
```

Richiede C++11 o successivo. Con `const Datum&` si legge ogni elemento senza copiarlo e senza modificarlo tramite d.

Attenzione: tramite un riferimento const si possono chiamare soltanto metodi const. Se `print()` o `value()` non sono dichiarati const, questo ciclo non compila.

```
for (Datum d : data) {
    // d e' una copia dell'elemento.
}

for (Datum& d : data) {
    // d permette di modificare l'elemento originale.
}
```

41 Vector come parametro di una funzione

Slide 114.

```
double mean(const std::vector<Datum>& data) {
    double somma = 0.0;

    for (const Datum& d : data) {
        somma += d.value(); // Richiede value() const.
    }

    return somma / data.size();
}
```

Condizione d'uso: il vector deve contenere almeno un elemento.

`const std::vector<Datum>&` evita di copiare il contenitore e impedisce alla funzione di modificarlo tramite questo parametro. La dimensione si ottiene dal contenitore: non occorre passare separatamente il numero di dati.

Per un header che dichiara queste funzioni, la soluzione semplice è includere:

```
#include <vector>
#include "Datum.h"
```

Non usare `Datum` senza renderne noto il nome tramite una definizione o un'appropriata dichiarazione anticipata.

Media e deviazione standard campionaria

```
#include <cmath>

Datum mean_and_stdDev(const std::vector<Datum>& data) {
    double m = mean(data);
    double sommaQuadrati = 0.0;

    for (const Datum& d : data) {
        double differenza = d.value() - m;
        sommaQuadrati += differenza * differenza;
    }

    double s = std::sqrt(
        sommaQuadrati / (data.size() - 1)
    );

    return Datum(m, s);
}
```

Condizione d'uso: almeno due elementi.

Come nella slide, questo calcolo usa i valori dei Datum e non le loro incertezze individuali. Restituisce la deviazione standard campionaria dei valori, non l'incertezza sulla media. Per dati indipendenti e identicamente distribuiti, quest'ultima si stima con $s/\sqrt{N}$.

42 Metodi const

Slide 116–117.

Il `const` dopo le parentesi qualifica il metodo rispetto all'oggetto sul quale opera.

```
// Dichiarazione nell'header.
double value() const;

// Definizione nel sorgente.
double Datum::value() const {
    return value_;
}
```

Un metodo const non può modificare i normali attributi dell'oggetto tramite `this`. Può comunque produrre effetti esterni, ad esempio stampare sul terminale.

```
Datum d1(1.2, 0.1);
const Datum d2(1.2, 0.1);

d1.value(); // Consentito.
d2.value(); // Consentito se value() e' const.

d1.setValue(2.0); // Consentito.
// d2.setValue(2.0); // Non consentito.
```

Correzione della slide 116: i metodi const possono essere chiamati sia su oggetti const sia su oggetti non const. I metodi non const non possono essere chiamati su oggetti const.

Getter e stampe sono normalmente buoni candidati per essere const. Setter che modificano gli attributi ordinari non lo sono.

43 Argomenti di default

Slide 118–120.

Un argomento di default viene usato quando il chiamante omette quel valore.

```
// Nell'header, dentro la classe Counter:
void increment(int step = 1);

// Nel sorgente:
void Counter::increment(int step) {
    count_ += step;
}
```

```
counter.increment();   // Usa step = 1.
counter.increment(14); // Usa step = 14.
```

Regola pratica: specifica il default nella dichiarazione visibile al chiamante e non ripeterlo nella definizione separata.

```
void fun(int i, int b = 0, int x = 3);

fun(3);       // Equivale a fun(3, 0, 3).
fun(3, 4);    // Equivale a fun(3, 4, 3).
fun(3, 4, 5); // Specifica tutti gli argomenti.
```

In questo uso ordinario, i parametri con default devono essere quelli finali: dopo un parametro con default, anche i successivi devono avere un default.

Attenzione: se adotti `increment(int step = 1)`, rimuovi l'altra versione `increment()`. Mantenerle entrambe renderebbe ambigua la chiamata senza argomenti.

Scegli default con un significato chiaro: per una quantità fisica, un valore arbitrario può nascondere un errore nell'uso della funzione.

44 Memoria dinamica: new e delete

Slide 122–126.

`new` crea un oggetto con durata dinamica e restituisce un puntatore. `delete` distrugge l'oggetto e libera la memoria associata.

```
double* p = new double(1.2);
std::cout << *p << std::endl;

delete p;
p = nullptr;
```

Per una classe, `new` chiama anche il costruttore:

```
Datum* p = new Datum(1.2, 0.3);
p->print();

delete p;
p = nullptr;
```

Il puntatore e l'oggetto puntato sono distinti. Uscire dall'ambito del puntatore locale non distrugge automaticamente l'oggetto creato con `new`.

Array dinamici

```
int n = 5; // Deve essere una dimensione valida.
double* dati = new double[n]{}; // Elementi a zero.

dati[0] = 1.2;

delete[] dati;
```

```
dati = nullptr;
```

Creazione	Rilascio
`new T(...)`	`delete p;`
`new T[n]`	`delete[] p;`

Attenzione: non mescolare `delete` e `delete[]`. Non liberare due volte lo stesso oggetto e non usarlo dopo averlo liberato. Assegnare `nullptr` a un puntatore non aggiorna eventuali altri puntatori allo stesso oggetto.

La forma ordinaria di `new` segnala un fallimento di allocazione tramite un'eccezione `std::bad_alloc`.

Memory leak e ownership

Un **memory leak** si verifica quando memoria allocata non viene più rilasciata, ad esempio perché si perde il puntatore necessario.

```
double* p = new double(1.2);
// Se perdiamo p senza delete, perdiamo l'accesso
// alla memoria da rilasciare.
```

Ownership: stabilisci chi è responsabile della durata e del rilascio della risorsa.

Prima di usare `new`, chiediti: serve davvero un oggetto dinamico? Chi lo distruggerà?

Per sequenze di dati, `std::vector` gestisce automaticamente la memoria e spesso evita la necessità di `new[]` manuale.

45 Distruttori e durata degli oggetti

Slide 127–132.

Il distruttore è una funzione speciale chiamata alla distruzione dell'oggetto.

```
class Counter {
public:
    Counter();
    ~Counter();
};
```

- Nome: tilde seguita dal nome della classe.
- Nessun tipo di ritorno.
- Nessun parametro.
- Serve a rilasciare le risorse possedute dall'oggetto.

Esempio di definizione nel sorgente:

```
Counter::~Counter() {
    std::cout << "Distruzione del Counter" << std::endl;
}
```

La stampa serve solo a osservare quando avviene la distruzione: non è necessaria per distruggere l'oggetto.

Oggetti automatici

```
{
    Datum d1(1.2, 0.3);
    Datum d2(2.0, 0.1);
}
// Distruzione: prima d2, poi d1.
```

Gli oggetti automatici vengono distrutti all'uscita dal proprio blocco. Nello stesso blocco, la distruzione avviene in ordine inverso al completamento della costruzione.

```
for (int i = 0; i < 3; ++i) {
    Datum d(1.2, 0.3);
} // d viene distrutto alla fine di ogni giro.
```

Oggetti dinamici

```
Datum* p = new Datum(1.2, 0.3);
delete p; // Chiama il distruttore e libera la memoria.
```

Precisazione: il distruttore esegue la distruzione e la pulizia delle risorse. Il rilascio della memoria dell'oggetto dinamico è gestito dall'espressione `delete`. Chiamare manualmente un distruttore non equivale a `delete`.

I membri oggetto, come `std::string` e `std::vector`, vengono distrutti automaticamente. Un membro puntatore grezzo non provoca da solo la distruzione dell'oggetto puntato.

Se una classe possiede memoria tramite puntatori grezzi, occorre progettare anche copia e assegnazione: una copia del solo indirizzo può causare rilasci duplicati o condivisioni involontarie.

Collegamenti simbolici nelle slide

```
ln -s ../../04/Datum.h Datum.h
```

Crea un collegamento simbolico al file indicato, non una copia indipendente. Il percorso relativo dipende dalla posizione del collegamento.

46 Overloading degli operatori

Slide 134–141.

L'overloading definisce il comportamento di operatori esistenti per tipi definiti dall'utente.

Dentro la classe Datum:

```
Datum operator+(const Datum& rhs) const;
```

Nel sorgente:

```
#include <cmath>

Datum Datum::operator+(const Datum& rhs) const {
    double val = value_ + rhs.value_;
    double err = std::sqrt(
        error_ * error_ + rhs.error_ * rhs.error_
    );

    return Datum(val, err);
}
```

```
Datum a(1.2, 0.3);
Datum b(-0.4, 0.4);

Datum c = a + b;
Datum d = a.operator+(b); // Stessa operazione.
```

- L'oggetto `a` è l'operando sinistro.
- `rhs` è l'operando destro.
- Il risultato viene restituito per valore.
- Il metodo const non modifica l'operando sinistro.
- Il parametro const protegge l'operando destro.

Regole: non si possono inventare nuovi operatori, cambiarne precedenza o numero di operandi, né ridefinire operazioni tra soli tipi built-in. Non tutti gli operatori sono sovraccaricabili: ad esempio `.`, `::`, `?:` e `sizeof` non lo sono.

Scegli comportamenti coerenti con il significato abituale dell'operatore.

47 Propagazione delle incertezze con Datum

Slide 137 e 152–153.

Gli esempi assumono incertezze standard non negative e dati non correlati. Per prodotto e rapporto si usa la propagazione al primo ordine.

Per $z = x + y$ oppure $z = x - y$:

$$\sigma_z = \sqrt{\sigma_x^2 + \sigma_y^2}.$$

Per $z = xy$:

$$\sigma_z = \sqrt{y^2\sigma_x^2 + x^2\sigma_y^2}.$$

Per $z = x/y$, con $y \neq 0$:

$$\sigma_z = \sqrt{\frac{\sigma_x^2}{y^2} + \frac{x^2\sigma_y^2}{y^4}}.$$

Per un fattore esatto k:

$$z = kx, \qquad \sigma_z = |k|\sigma_x.$$

Se i dati sono correlati, occorre includere i termini di covarianza. Usare due volte lo stesso Datum non rende le due occorrenze statisticamente indipendenti.

Esempio: `a * a` con l'operatore seguente dà $\sqrt{2}\,|x|\,\sigma_x$, mentre l'incertezza corretta di x^2 è $2|x|\sigma_x$. Allo stesso modo, `a - a` dà $\sqrt{2}\,\sigma_x$ invece di zero.

Prodotto tra Datum

Da dichiarare nella classe:

```
Datum operator*(const Datum& rhs) const;
```

Nel sorgente:

```
Datum Datum::operator*(const Datum& rhs) const {
    double val = value_ * rhs.value_;
    double err = std::sqrt(
        rhs.value_ * rhs.value_ * error_ * error_
        + value_ * value_ * rhs.error_ * rhs.error_
    );

    return Datum(val, err);
}
```

Rapporto tra Datum

Da dichiarare nella classe:

```
Datum operator/(const Datum& rhs) const;
```

Nel sorgente:

```
Datum Datum::operator/(const Datum& rhs) const {
    // Richiede rhs.value_ != 0.
    double val = value_ / rhs.value_;
    double termine1 = error_ / rhs.value_;
    double termine2 = val * rhs.error_ / rhs.value_;

    double err = std::sqrt(
```

```
        termine1 * termine1 + termine2 * termine2
    );

    return Datum(val, err);
}
```

Questa forma evita di dividere per `value_`: può essere usata anche con numeratore nullo. La formula relativa della slide richiede invece attenzione quando il numeratore è zero.

48 this e restituzione dell'oggetto corrente

Slide 145–149.

`this` è un puntatore all'oggetto corrente; `*this` è l'oggetto stesso.

```
// Dentro un metodo non statico:
std::cout << this << std::endl;
```

In un metodo const di Datum, `this` ha tipo `const Datum*`: tramite esso non si possono modificare gli attributi. Un metodo statico non dispone di `this`.

Gli operatori che modificano l'oggetto possono restituire un riferimento a `*this`.

49 Copia e assegnazione

Slide 142–144 e 148.

```
Datum a(1.2, 0.3);

Datum b = a; // Costruisce b: inizializzazione per copia.

Datum c(0.0, 0.0);
c = a;       // Modifica c: assegnazione.
```

Il simbolo `=` in una dichiarazione non indica una chiamata all'operatore di assegnazione: `Datum b = a;` usa il costruttore di copia.

Operatore di assegnazione

Nella classe:

```
Datum& operator=(const Datum& rhs);
```

Nel sorgente:

```
Datum& Datum::operator=(const Datum& rhs) {
    value_ = rhs.value_;
    error_ = rhs.error_;

    return *this;
}
```

Uso: modifica l'operando sinistro e restituisce un riferimento a quell'oggetto. Il metodo non è const perché modifica gli attributi.

Per questa classe semplice, l'assegnazione generata dal compilatore copia già i membri: la versione esplicita serve a illustrare la sintassi.

Le slide restituiscono spesso `const Datum&`; la forma convenzionale è `Datum&`. Il const sul tipo restituito limita le operazioni tramite il riferimento restituito: non impedisce al metodo di modificare l'oggetto.

50 Assegnazione composta: operator+=

Slide 150 e 161.

Definire `+` e `=` non definisce automaticamente `+=`.

Nella classe:

```
Datum& operator+=(const Datum& rhs);
```

Nel sorgente:

```
Datum& Datum::operator+=(const Datum& rhs) {
    value_ += rhs.value_;
    error_ = std::sqrt(
        error_ * error_ + rhs.error_ * rhs.error_
    );

    return *this;
}
```

```
a += b; // Modifica a.
```

Errore da evitare: creare e restituire un nuovo oggetto senza modificare `*this` non realizza l'assegnazione desiderata.

Restituire per valore non impedisce, di per sé, di modificare l'operando sinistro. Nell'esempio errato della slide 161, la modifica manca nel corpo dell'operatore. Restituire per riferimento mantiene invece il collegamento all'oggetto modificato.

Non restituire un riferimento a una variabile locale: verrebbe distrutta all'uscita dalla funzione.

51 Confronti tra oggetti

Slide 151.

Nella classe:

```
bool operator<(const Datum& rhs) const;
```

Nel sorgente:

```
bool Datum::operator<(const Datum& rhs) const {
    return value_ < rhs.value_;
}
```

```
if (a < b) {
    std::cout << "a ha valore centrale minore";
}
```

Questo confronto ordina soltanto i valori centrali: non valuta la compatibilità statistica.

Attenzione: se `a < b` è falso, non segue necessariamente `b < a`: i valori possono essere uguali. Definire `<` non definisce automaticamente `>`, `<=` o `==`.

52 Operatori membri e funzioni libere

Slide 153–157.

Un operatore membro usa l'oggetto corrente come operando sinistro.

Nella classe:

```
Datum operator*(double k) const;
```

Nel sorgente:

```
#include <cmath> // Per std::abs con double.

Datum Datum::operator*(double k) const {
    return Datum(value_ * k, error_ * std::abs(k));
}
```

Attenzione: usa `std::abs` includendo `<cmath>`. La funzione `abs` senza `std::` può corrispondere alla versione per interi e troncare il fattore: ad esempio 0.5 diventerebbe 0.

Questo permette:

```
Datum risultato = a * 0.5;
```

Per l'ordine opposto, si può definire una funzione libera. Dichiarazione nell'header, fuori dalla classe:

```
Datum operator*(double k, const Datum& rhs);
```

Definizione nel sorgente:

```
Datum operator*(double k, const Datum& rhs) {
    return rhs * k;
}
```

```
Datum risultato = 0.5 * a;
```

Senza questa funzione libera, `0.5 * a` non compila: l'operatore membro richiede un Datum come operando sinistro.

Attenzione: l'incertezza deve essere moltiplicata per il valore assoluto del fattore, anche quando `k` è negativo. Le slide mostrano soltanto fattori positivi.

53 Stampare oggetti con operator«

Slide 158.

Nell'header, fuori dalla classe:

```
#include <ostream>

std::ostream& operator<<(std::ostream& os,
                         const Datum& rhs);
```

Nel sorgente:

```
std::ostream& operator<<(std::ostream& os,
                         const Datum& rhs) {
    os << rhs.value() << " +/- " << rhs.error();
    return os;
}
```

```
std::cout << a << " ; " << b << std::endl;
```

Regole: scrivi nel flusso `os` ricevuto, non direttamente in `std::cout`. Restituisci `os` per permettere concatenazioni. Lascia al chiamante la scelta di andare a capo.

I getter devono essere const perché `rhs` è un riferimento a un oggetto const.

54 friend: accesso autorizzato ai membri privati

Slide 159–160.

Una funzione libera non accede normalmente agli attributi privati. La classe può autorizzarla con `friend`.

Dentro la classe:

```
friend std::ostream& operator<<(std::ostream& os,
                                const Datum& rhs);
```

Nel sorgente:

```
std::ostream& operator<<(std::ostream& os,
                         const Datum& rhs) {
    os << rhs.value_ << " +/- " << rhs.error_;
    return os;
}
```

La definizione non usa `Datum::`: la funzione resta libera, non è un metodo. Questa è un'alternativa alla versione con getter: non definire entrambe le versioni.

Da ricordare: una funzione friend non diventa un metodo della classe e non riceve un puntatore `this`. L'autorizzazione permette l'accesso ai membri privati.

Non usare friend soltanto per presumere un vantaggio di velocità: getter brevi possono essere ottimizzati dal compilatore.

55 Schema rapido degli operatori

Operazione	Effetto abituale	Risultato abituale
`a + b`	Crea un risultato senza modificare a e b.	Oggetto per valore.
`a * b`	Crea un risultato senza modificare a e b.	Oggetto per valore.
`a = b`	Sostituisce il contenuto di a.	Riferimento ad a.
`a += b`	Modifica a accumulando b.	Riferimento ad a.
`a < b`	Confronta secondo il criterio definito.	bool.
`os « a`	Scrive a nel flusso os.	Riferimento a os.

56 Attributi static: dati condivisi tra oggetti

Slide 163–169.

Un attributo ordinario appartiene al singolo oggetto. Un attributo `static` appartiene alla classe: esiste una sola variabile condivisa tra tutte le istanze.

```
// Nell'header Unit.h:
class Unit {
public:
    Unit();
    ~Unit();

    static int getCount();

private:
    static int counter_;
};
```

```
// Nel sorgente Unit.cc:
#include "Unit.h"

// Definizione unica dell'attributo static.
// Non si ripete la parola static.
int Unit::counter_ = 0;

Unit::Unit() {
    ++counter_;
}

Unit::~Unit() {
    --counter_;
}
```

Dove va la definizione: nell'header c'è soltanto la dichiarazione; la riga `int Unit::counter_ = 0;` va nel sorgente, una sola volta. Se la scrivi nell'header e l'header è incluso da più sorgenti, il linker segnala una definizione multipla. Se la dimentichi, il linker segnala un simbolo non definito.

Significato del contatore: incrementando alla costruzione e decrementando alla distruzione, si contano gli oggetti attualmente esistenti, non tutti quelli creati dall'inizio del programma.

Attenzione alla copia: il costruttore di copia generato dal compilatore non esegue il corpo di `Unit()`. Se vogliamo contare correttamente anche le copie, dobbiamo gestirle:

```
// Dichiarazione nella classe:
Unit(const Unit& other);
```

```
// Definizione nel sorgente.
// Il parametro non e' usato: si puo' omettere il nome.
Unit::Unit(const Unit&) {
    ++counter_;
}
```

Qui `Unit` non contiene attributi ordinari da copiare. Se ne avesse, andrebbero copiati correttamente. Ogni percorso di costruzione di un nuovo oggetto deve aggiornare il contatore.

L'assegnazione non crea un nuovo oggetto e quindi non deve incrementare il numero di oggetti esistenti.

57 Metodi static

Slide 170–172 e 175.

Un metodo `static` si può chiamare senza creare un oggetto della classe.

```
// Dichiarazione nella classe:
static int getCount();

// Definizione nel sorgente:
int Unit::getCount() {
    return counter_;
}
```

```
// Dentro main:
std::cout << Unit::getCount(); // 0: anche senza oggetti.

{
    Unit a;
    Unit b;
    std::cout << Unit::getCount(); // 2.
}

std::cout << Unit::getCount(); // 0.
```

Regole:

- `static` compare nella dichiarazione dentro la classe, non nella definizione esterna.
- Il metodo non dispone di `this`.
- Non può avere il qualificatore `const` dopo le parentesi.
- Può accedere direttamente agli attributi static.
- Per accedere a un attributo ordinario deve avere un oggetto esplicitamente disponibile.

```
// Esempio dentro una classe con attributo name_:
static void rename(Unit& u) {
    u.name_ = "Nuovo nome"; // Accesso tramite un oggetto.
}
```

Anche se è possibile chiamare un metodo static tramite un oggetto, la forma `Unit::getCount()` rende evidente che l'operazione riguarda la classe.

58 Creazione con nome: funzioni factory

Slide 173–174.

Due costruttori con gli stessi tipi di parametri non possono essere distinti soltanto dal significato dei numeri o dai nomi dei parametri.

Per distinguere coordinate cartesiane e polari, si possono usare funzioni static con nomi espliciti.

```
#include <cmath>
```

```
class Point {
public:
    static Point Cartesian(double x, double y) {
        return Point(x, y);
    }

    static Point Polar(double r, double theta) {
        return Point(r * std::cos(theta),
                     r * std::sin(theta));
    }

    double x() const { return x_; }
    double y() const { return y_; }

private:
    Point(double x, double y) {
        x_ = x;
        y_ = y;
    }

    double x_;
    double y_;
};
```

```
Point a = Point::Cartesian(3.0, 4.0);
Point b = Point::Polar(5.0, 0.5);
```

A cosa serve: rendere esplicito come interpretare gli argomenti. Le funzioni static restituiscono oggetti per valore e possono chiamare il costruttore privato.

Attenzione: `std::sin` e `std::cos` ricevono angoli in radianti. Questo esempio conserva internamente sempre coordinate cartesiane.

Stesso problema in esame: un costruttore che riceve "la carica *oppure* la tensione" non si può scrivere come due costruttori con gli stessi tipi. Funzioni factory con nomi diversi, ad esempio `fromCharge(...)` e `fromVoltage(...)`, risolvono l'ambiguità.

59 Classi di utilità

Slide 176–180.

Una classe può raggruppare algoritmi come metodi static, senza richiedere un oggetto per eseguirli.

```
// Calculator.h
#ifndef CALCULATOR_H
#define CALCULATOR_H

#include <vector>
#include "Datum.h"

class Calculator {
public:
    static Datum weightedAverage(
        const std::vector<Datum>& data
    );

    static Datum arithmeticAverage(
        const std::vector<Datum>& data
    );

    static Datum geometricAverage(
        const std::vector<Datum>& data
    );
};

#endif
```

Le dichiarazioni mostrano l'interfaccia: ogni metodo usato deve avere una definizione.

```
// Nel sorgente (schema):
Datum Calculator::weightedAverage(
    const std::vector<Datum>& data
) {
    // Calcolo...
    // return Datum(valore, incertezza);
}
```

Questo è soltanto uno schema: il corpo deve calcolare e restituire un Datum. Senza `return`, il comportamento è indefinito.

Separare le responsabilità

Classe	Compito
`Datum`	Conserva valore e incertezza.
`InputService`	Acquisisce i dati dell'utente.
`Calculator`	Esegue i calcoli sui dati.

```
std::vector<Datum> data =
    InputService::readDataFromUser();

Datum risultato = Calculator::weightedAverage(data);
std::cout << risultato << std::endl;
```

Queste chiamate presuppongono che le classi e i metodi siano stati dichiarati e implementati.

Compilare il progetto

```
clang++ -std=c++17 -Wall -c Datum.cc
clang++ -std=c++17 -Wall -c Calculator.cc
clang++ -std=c++17 -Wall -c InputService.cc

clang++ -std=c++17 -Wall wgtavg.cpp Datum.o Calculator.o InputService.o -o wgtavg
```

Senza `-o`, l'opzione `-c` produce un file oggetto con lo stesso nome del sorgente ed estensione `.o`.

60 Applicazione: media pesata

Applicazione delle slide 177–180.

Per misure indipendenti della stessa grandezza, con incertezze standard note e positive:

$$w_i = \frac{1}{\sigma_i^2}, \qquad \bar{x}_w = \frac{\sum_i w_i x_i}{\sum_i w_i}, \qquad \sigma_{\bar{x}_w} = \frac{1}{\sqrt{\sum_i w_i}}.$$

Un'implementazione essenziale dell'algoritmo:

```
#include <cmath>

Datum Calculator::weightedAverage(
    const std::vector<Datum>& data
) {
    double sommaPesi = 0.0;
    double sommaPesata = 0.0;

    for (const Datum& d : data) {
        double sigma = d.error();
        double peso = 1.0 / (sigma * sigma);

        sommaPesi += peso;
        sommaPesata += peso * d.value();
    }
```

```
    return Datum(
        sommaPesata / sommaPesi,
        std::sqrt(1.0 / sommaPesi)
    );
}
```

Condizioni d'uso: contenitore non vuoto, incertezze positive e valori numericamente adatti al calcolo. Il codice non verifica queste condizioni.

I getter `value()` ed `error()` devono essere const. La presenza di correlazioni richiede una formula che includa la matrice di covarianza.

Le slide propongono anche media aritmetica e geometrica: prima di implementarle, occorre specificare la definizione e il significato dell'incertezza restituita. La media geometrica ordinaria richiede valori positivi.

61 Tolleranza condivisa e operator==

Slide 165 e 181–182.

Una tolleranza static permette di usare la stessa soglia per tutti gli oggetti.

Da aggiungere alla classe Datum:

```
public:
    static void setTolerance(double val);
    bool operator==(const Datum& rhs) const;
    bool operator!=(const Datum& rhs) const;

private:
    static double tolerance_;
```

Nel sorgente Datum.cc:

```
#include <cmath>

double Datum::tolerance_ = 1e-4;

void Datum::setTolerance(double val) {
    // Richiede val > 0 e finito.
    tolerance_ = val;
}

bool Datum::operator==(const Datum& rhs) const {
    return std::abs(value_ - rhs.value_) < tolerance_
        && std::abs(error_ - rhs.error_) < tolerance_;
}

bool Datum::operator!=(const Datum& rhs) const {
    return !(*this == rhs); // Riusa operator==.
}
```

```
Datum::setTolerance(0.01);

if (a == b) {
    std::cout << "Uguali secondo la tolleranza";
}
```

`&&` è l'AND logico: entrambe le condizioni devono essere vere. Se la prima è falsa, la seconda non viene valutata.

Attenzione: fino a C++17, definire `==` non definisce automaticamente `!=`: serve l'operatore apposito.

Significato: si confrontano separatamente valori centrali e incertezze con una soglia assoluta. Non è un test di compatibilità statistica.

Cambiare la tolleranza modifica il criterio per tutti gli oggetti, anche quelli già esistenti. La soglia deve essere coerente con le unità utilizzate.

Attenzione: la disuguaglianza è stretta: una differenza esattamente pari alla tolleranza non è accettata. I numeri decimali non sono tutti rappresentabili esattamente in virgola mobile.

L'uguaglianza approssimata non è generalmente transitiva: `a` può risultare vicino a `b` e `b` vicino a `c`, senza che `a` risulti vicino a `c`. Non trattarla automaticamente come un'uguaglianza matematica esatta.

62 Formattare i numeri: manipolatori di output

Slide 183.

```
#include <iostream>
#include <iomanip>

double x = 1.23456789;

std::cout << std::defaultfloat
          << std::setprecision(3) << x << '\n';

std::cout << std::fixed
          << std::setprecision(3) << x << '\n';

std::cout << std::scientific
          << std::setprecision(3) << x << '\n';
```

Manipolatore	Effetto
`defaultfloat`	Formato generale; `setprecision` indica le cifre significative.
`fixed`	Notazione decimale fissa; `setprecision` indica le cifre dopo la virgola.
`scientific`	Notazione scientifica; `setprecision` indica le cifre dopo la virgola nella mantissa.

Per l'esempio, l'output è:

```
1.23
1.235
1.235e+00
```

Attenzione: queste impostazioni rimangono nel flusso per le stampe successive finché non vengono cambiate. Modificano la rappresentazione stampata, non il valore conservato nella variabile.

Per ripristinare formato generale e precisione 6:

```
std::cout << std::defaultfloat
          << std::setprecision(6);
```

Un `operator«` che scrive i valori nel flusso ricevuto rispetta normalmente queste impostazioni.

63 Schema rapido: ordinario oppure static

	Ordinario	Static
Attributo	Un valore per ogni oggetto.	Un valore condiviso dalla classe.
Definizione	Nella classe.	Dichiarato nella classe, definito una volta nel sorgente.
Metodo	Richiede un oggetto.	Chiamabile tramite `Classe::metodo()`.
`this`	Disponibile nei metodi non static.	Non disponibile.
`const` finale	Ammesso per un metodo non static.	Non ammesso.

64 std::pair: associare due valori

Slide 186–188.

Una `pair` contiene due elementi, anche di tipi diversi. Si accede agli elementi con `first` e `second`.

```
#include <utility>
#include <string>

std::pair<std::string, int> voto =
```

```
    std::make_pair("RQM", 24);

std::cout << voto.first;  // Nome dell'esame.
std::cout << voto.second; // Voto.
```

Un vector di pair conserva una sequenza di coppie:

```
#include <vector>

std::vector<std::pair<std::string, int>> grades;

grades.push_back(std::make_pair("RQM", 26));
grades.push_back(std::make_pair("Phys Lab", 27));

for (const auto& grade : grades) {
    std::cout << grade.first << ": "
              << grade.second << '\n';
}
```

`auto` fa dedurre il tipo al compilatore. Qui `const auto&` permette di leggere ogni coppia senza copiarla.

65 std::map: associare chiavi e valori

Slide 185 e 187–188.

Una `map` associa a ogni chiave un valore. Le chiavi sono uniche secondo il criterio di confronto e vengono mantenute ordinate.

```
#include <map>
#include <string>

std::map<std::string, int> voti;

voti["RQM"] = 26;
voti["Phys Lab"] = 27;

std::cout << voti.at("RQM"); // 26.
```

Operazione	Effetto
`m[chiave]`	Accede al valore; se la chiave manca, inserisce una nuova voce con valore inizializzato.
`m.at(chiave)`	Accede senza inserire; se la chiave manca, genera un'eccezione.
`m.find(chiave)`	Restituisce un iteratore alla voce, oppure `end()` se non esiste.
`m.size()`	Numero di associazioni.

```
auto it = voti.find("RQM");

if (it != voti.end()) {
    std::cout << it->first << ": " << it->second;
}
```

Attenzione: `operator[]` non è una semplice lettura: può modificare la map inserendo una voce. Non è utilizzabile su una map const.

Le operazioni di ricerca e inserimento nella `std::map` hanno complessità logaritmica nel numero di elementi.

66 Usare una classe come chiave

Slide 187–188.

La map deve poter ordinare le chiavi. Con il comparatore predefinito, possiamo fornire `operator<` nella classe.

```
#include <string>

class Examinee {
public:
```

```
    Examinee(const std::string& name, int id) {
        name_ = name;
        id_ = id;
    }

    bool operator<(const Examinee& rhs) const {
        return id_ < rhs.id_;
    }

    std::string name() const { return name_; }
    int id() const { return id_; }

private:
    std::string name_;
    int id_;
};
```

Per questa classe, la map distingue le chiavi in base alla matricola, non al nome. Due oggetti con la stessa matricola sono chiavi equivalenti anche con nomi diversi.

In generale, due chiavi `a` e `b` sono equivalenti per la map se entrambi i confronti `a < b` e `b < a` sono falsi. Il criterio deve rispettare un ordinamento debole stretto.

Attenzione: `operator<` deve essere un metodo const (o una funzione libera con parametri const): la map confronta chiavi const.

Esempio: studente e lista di esami

```
using Grade = std::pair<std::string, int>;
using Grades = std::vector<Grade>;

std::map<Examinee, Grades> exams;

Examinee tom("Tom", 110998);

Grades grades;
grades.push_back(std::make_pair("RQM", 26));
grades.push_back(std::make_pair("Phys Lab", 27));

exams[tom] = grades;
```

`using` introduce qui un alias di tipo: non è una direttiva `using namespace`.

L'assegnazione copia il vector nella map. Svuotare successivamente `grades` non svuota la copia conservata in `exams`.

```
grades.clear(); // Rimuove tutti gli elementi.
```

`clear()` non garantisce il rilascio della capacità di memoria del vector.

Attraversare i due livelli

```
for (const auto& entry : exams) {
    const Examinee& student = entry.first;
    const Grades& studentGrades = entry.second;

    std::cout << student.name() << " "
              << student.id() << '\n';

    for (const auto& grade : studentGrades) {
        std::cout << grade.first << ": "
                  << grade.second << '\n';
    }
}
```

Da ricordare: `first` è la chiave della map; `second` è il valore associato. All'interno del vector, `first` e `second` sono invece nome dell'esame e voto.

La chiave conservata nella map è const: non si può modificarla direttamente tramite l'iteratore.

67 Ereditarietà: classe base e classe derivata

Slide 193–197.

L'ereditarietà permette di definire un tipo che estende un tipo esistente. Con ereditarietà pubblica, `Student` rappresenta un caso particolare di `Person`.

```
#include <string>

class Person {
public:
    Person(const std::string& name) {
        name_ = name;
    }

    std::string name() const {
        return name_;
    }

private:
    std::string name_;
};

class Student : public Person {
public:
    Student(const std::string& name, int id)
        : Person(name) {
        id_ = id;
    }

    int id() const {
        return id_;
    }

private:
    int id_;
};
```

```
Student s("Susan", 123456);

std::cout << s.name(); // Metodo della base.
std::cout << s.id();   // Metodo della derivata.
```

Ogni `Student` contiene una parte `Person` e i membri aggiunti da `Student`. La derivata non può accedere direttamente agli attributi privati della base: qui usa il getter `name()`.

68 Costruzione e distruzione della derivata

Slide 194–195 e 201–203.

```
Student(const std::string& name, int id)
    : Person(name) {
    id_ = id;
}
```

La parte dopo i due punti è la lista di inizializzazione del costruttore. Qui seleziona il costruttore della base.

Inizializzare anche i membri nella lista

La lista può inizializzare anche gli attributi, non solo la base:

```
Student(const std::string& name, int id)
```

```
    : Person(name), id_(id) {
    // Corpo vuoto: tutto e' gia' inizializzato.
}
```

La lista è **obbligatoria** per: la base senza costruttore di default, gli attributi `const` e gli attributi riferimento. Per gli altri attributi è comunque la forma consigliata.

I membri vengono inizializzati nell'ordine in cui sono *dichiarati* nella classe, non nell'ordine in cui compaiono nella lista. Scrivi la lista nello stesso ordine per evitare avvisi e confusione.

Ordine di costruzione: prima la base, poi i membri della derivata nel loro ordine di dichiarazione, infine il corpo del costruttore della derivata.

Se non specifichiamo il costruttore della base, viene tentata la sua inizializzazione di default. Se non è disponibile un costruttore di default accessibile, il codice non compila.

Anche quando esiste, il costruttore di default può non impostare i dati desiderati: un nome passato al costruttore di `Student` non viene automaticamente passato a `Person`.

Distruzione: prima viene eseguito il corpo del distruttore della derivata; poi vengono distrutti i suoi membri e la parte base. Non si chiama manualmente il distruttore della base.

69 public, protected e private

Slide 199–200.

Un membro `protected` è accessibile dalla classe e dalle classi derivate, ma non dal normale codice esterno.

```
class Person {
protected:
    int age_ = 0; // Inizializzazione nella dichiarazione.
};

class Student : public Person {
public:
    void birthday() {
        ++age_; // Accesso consentito alla derivata.
    }
};
```

`int age_ = 0;` nella classe assegna un valore iniziale predefinito all'attributo (C++11 o successivo). Senza di esso, `++age_` leggerebbe un valore non inizializzato.

Effetto del tipo di ereditarietà

Membro nella base	public	protected	private
public	public	protected	private
protected	protected	protected	private
private	Non accessibile	Non accessibile	Non accessibile

Le tre colonne indicano il tipo di ereditarietà. I membri privati della base rimangono parte dell'oggetto, ma non sono direttamente accessibili dalla derivata.

Attenzione: per una `class`, omettere lo specificatore di ereditarietà significa ereditarietà private. Scrivi esplicitamente `public` quando vuoi la relazione pubblica "è un".

70 Conversione dalla derivata alla base

Slide 198.

Con ereditarietà pubblica, un puntatore o riferimento alla derivata può essere usato come puntatore o riferimento alla base.

```
Student s("Susan", 123456);

Person* p = &s;
Person& r = s;
```

```
std::cout << p->name(); // Consentito.
// p->id();            // Non consentito.
```

Il tipo del puntatore `p` è `Person*`: tramite esso sono disponibili i membri dichiarati nell'interfaccia di `Person`.

La conversione inversa non è automaticamente valida: una `Person` non è necessariamente uno `Student`.

71 Metodi omonimi e chiamate non virtuali

Slide 204–207.

Se la derivata definisce un metodo con lo stesso nome di un metodo non virtuale della base, il metodo della base viene nascosto. Non si tratta di dispatch dinamico.

Esempio da integrare nelle due classi:

```
// Nella classe Person:
void print() const {
    std::cout << "Person: " << name() << '\n';
}

// Nella classe Student:
void print() const {
    std::cout << "Student: " << name()
              << " id: " << id() << '\n';
}
```

```
Student s("Susan", 123456);
Person* p = &s;

s.print();         // Student::print().
p->print();        // Person::print().
s.Person::print(); // Richiede esplicitamente la base.
```

Per una chiamata non virtuale, la selezione dipende dal tipo statico dell'espressione, noto in compilazione.

Attenzione: l'hiding avviene anche se i parametri sono diversi. Se `Student` definisse soltanto `print(int n)`, la chiamata `s.print()` non compilerebbe: il nome `print` della derivata nasconde tutte le versioni della base.

Distinzione terminologica: overloading significa stesso nome con parametri diversi nello stesso ambito. Ridefinire nella derivata un metodo non virtuale come nell'esempio comporta hiding. L'overriding polimorfico richiede un metodo virtuale nella base, argomento successivo del corso.

72 Scelta a runtime: if, else e do-while

Slide 207.

Un blocco `else` viene eseguito quando la condizione dell'`if` è falsa:

```
if (scelta == 1) {
    // Caso Person.
} else {
    // Altro caso.
}
```

Un ciclo `do-while` esegue il corpo almeno una volta e controlla la condizione alla fine.

```
int scelta = 0;

do {
    std::cout << "1: Person, 2: Student\n";

    if (!(std::cin >> scelta)) {
        return 1; // Qui siamo dentro main.
    }
} while (scelta != 1 && scelta != 2);
```

Il punto e virgola dopo la condizione del `do-while` è necessario. Il controllo dell'input evita di continuare a ripetere la lettura su un flusso in errore.

Scegliere a runtime quale oggetto creare non basta a rendere virtuali i suoi metodi: un `Person*` chiama ancora `Person::print()` se quel metodo non è virtuale.

73 Distruzione tramite puntatore alla base

Slide 207–208: precisazione essenziale.

Se vogliamo distruggere un oggetto derivato tramite un puntatore alla base, la base deve avere un distruttore virtuale.

```
// Da aggiungere nella sezione public di Person:
virtual ~Person() = default;
```

```
Person* p = new Student("Susan", 123456);
delete p;
```

Con il distruttore virtuale, viene distrutto correttamente l'intero oggetto derivato.

Attenzione alla slide 207: senza distruttore virtuale nella base, questa `delete` produce comportamento indefinito. Non è soltanto una mancata stampa del distruttore della derivata.

La sintassi `= default` richiede al compilatore di fornire l'implementazione predefinita. Un distruttore predefinito può quindi essere virtuale.

Questo anticipa un requisito necessario per usare in sicurezza l'esempio; il polimorfismo verrà approfondito successivamente.

74 Schema rapido dell'ereditarietà

Sintassi	Significato
`class D : public B`	D deriva pubblicamente da B.
`D(...) : B(...), x_(x)`	Il costruttore di D inizializza la parte B e i propri membri.
`B* p = &d;`	Accesso a un oggetto derivato tramite la base.
`d.B::metodo()`	Chiamata esplicita alla versione della base.
`protected:`	Accesso consentito anche alle classi derivate.
`virtual ~B() = default;`	Distruzione corretta della derivata tramite B*.

75 Polimorfismo

Slide 209–226.

Il polimorfismo permette di usare un'interfaccia comune per oggetti che possono comportarsi in modo diverso.

- **Polimorfismo statico**: la scelta avviene durante la compilazione, come nell'overloading di funzioni e operatori.
- **Polimorfismo dinamico**: la scelta avviene durante l'esecuzione, usando ereditarietà e metodi virtuali.

Tipo statico e tipo dinamico

```
Person* p = new Student("Susan", 123456);
```

- Il tipo statico di `p` è `Person*`: è quello dichiarato nel codice.
- Il tipo dinamico dell'oggetto puntato è `Student`: è il tipo dell'oggetto effettivamente creato.

Un puntatore alla classe base può puntare a un oggetto di una classe derivata, se l'ereditarietà è pubblica. La conversione inversa non avviene automaticamente.

Metodi virtuali e override

Un metodo dichiarato `virtual` nella classe base può essere ridefinito nelle classi derivate.

Quando viene chiamato attraverso un puntatore o un riferimento alla classe base, viene scelta la versione corrispondente al tipo dinamico dell'oggetto.

```
#include <iostream>
#include <string>

class Person {
public:
    explicit Person(const std::string& name)
        : name_(name)
    {
    }

    // Necessario per distruggere oggetti derivati
    // attraverso un puntatore Person*.
    virtual ~Person() = default;

    virtual void print() const
    {
        std::cout << "Person: " << name_ << '\n';
    }

    const std::string& name() const
    {
        return name_;
    }

private:
    std::string name_;
};

class Student : public Person {
public:
    Student(const std::string& name, int id)
        : Person(name), id_(id)
    {
    }

    // override controlla che questo metodo
    // ridefinisca un metodo virtuale della base.
    void print() const override
    {
        std::cout << "Student: " << name()
                  << ", id: " << id_ << '\n';
    }

    int id() const
    {
        return id_;
    }

private:
    int id_;
};

class GraduateStudent : public Student {
public:
    GraduateStudent(const std::string& name,
                    int id,
                    const std::string& major)
        : Student(name, id), major_(major)
    {
    }
```

```
    void print() const override
    {
        std::cout << "GraduateStudent: " << name()
                  << ", id: " << id()
                  << ", major: " << major_ << '\n';
    }

private:
    std::string major_;
};
```

`explicit` impedisce le conversioni implicite tramite quel costruttore: ad esempio, una stringa non diventa automaticamente una `Person`.

Da ricordare:

- La firma deve corrispondere a quella della base: anche il `const` finale conta.
- Una volta virtuale nella base, il metodo resta virtuale nelle derivate.
- `override` non è obbligatorio, ma permette di individuare errori nella firma.
- Se una derivata non ridefinisce il metodo, eredita la versione disponibile nella gerarchia.
- Se si separano header e implementazione, `virtual`, `override` e `= 0` si scrivono soltanto nella dichiarazione dentro la classe. Ripeterli nella definizione esterna è un errore di compilazione.

```
// Student.h, dentro la classe:
void print() const override;

// Student.cc: senza virtual e senza override.
void Student::print() const
{
    std::cout << "Student: " << name()
              << ", id: " << id_ << '\n';
}
```

Chiamata attraverso puntatore e riferimento

Il seguente codice usa le classi appena definite.

```
int main()
{
    Student susan("Susan", 123456);

    Person* p = &susan;
    p->print();       // Chiama Student::print().

    Person& r = susan;
    r.print();        // Chiama Student::print().

    (*p).print();     // Equivale a p->print().

    return 0;
}
```

Precisazione sulle slide: il punto `.` non impedisce il polimorfismo. Anche `r.print()` e `(*p).print()` possono effettuare una chiamata virtuale.

Il tipo statico determina quali membri sono accessibili. Il tipo dinamico determina quale versione del metodo virtuale viene eseguita.

```
Student susan("Susan", 123456);
Person* p = &susan;

p->print(); // Valido: print() esiste in Person.

// p->id(); // Errore: Person non dichiara id().
```

Un metodo della base che usa un metodo virtuale

Un metodo comune, scritto una sola volta nella base, può chiamare un metodo virtuale: viene eseguita la versione della classe derivata.

```
// Dentro la classe base Function (vedi sotto):
double mean(double a, double b) const
{
    // integrate() e' virtuale: usa la versione
    // della derivata effettiva.
    return integrate(a, b) / (b - a);
}
```

Questo schema evita di ripetere in ogni derivata le formule che dipendono soltanto dal risultato del metodo virtuale.

Chiamate virtuali nei costruttori: da evitare

Durante l'esecuzione del costruttore della base, la parte derivata non è ancora stata costruita: l'oggetto si comporta come un oggetto della base.

- Una chiamata a un metodo virtuale nel costruttore (o nel distruttore) della base esegue la versione della base, non quella della derivata.
- Se il metodo è virtuale puro, la chiamata produce comportamento indefinito o un errore di linking.

Soluzione: nel costruttore conserva soltanto i dati ricevuti. Calcola le grandezze che dipendono dal metodo virtuale nei metodi chiamati dopo la costruzione, ad esempio in un getter.

Stampa polimorfica con operator«

`operator«` non può essere virtuale, perché è una funzione libera. Si definisce una sola volta per la base e si fa chiamare un metodo virtuale:

```
// Nella base, dentro la classe:
virtual void print(std::ostream& os) const;

// Nell'header, fuori dalla classe:
std::ostream& operator<<(std::ostream& os,
                         const Person& p);

// Nel sorgente:
std::ostream& operator<<(std::ostream& os,
                         const Person& p)
{
    p.print(os); // Chiamata virtuale.
    return os;
}
```

Ogni derivata ridefinisce `print(std::ostream&) const` con `override` e scrive in `os`. Allora `std::cout « susan` stampa la versione corretta per qualsiasi derivata.

Attenzione alla copia per valore

```
Student susan("Susan", 123456);

Person copia = susan;
// Copia soltanto la parte Person.
// I dati specifici di Student vengono persi.

copia.print(); // L'oggetto copia e' una Person.
```

Questo fenomeno si chiama **object slicing**. Per conservare il comportamento polimorfico, si usano puntatori o riferimenti alla classe base. Lo stesso vale per i parametri delle funzioni: usa `const Person&`, non `Person`.

Collezione di oggetti diversi

```
#include <vector>

// Dentro main(), usando le classi precedenti:
Person john("John");
Student susan("Susan", 123456);
GraduateStudent paolo("Paolo", 9856, "Physics");

std::vector<Person*> people{
    &john, &susan, &paolo
};

for (const Person* p : people) {
    p->print(); // Versione scelta in base all'oggetto.
}

// Non usare delete: questi oggetti sono automatici.
// Il vettore contiene soltanto i loro indirizzi.
```

Un `std::vector<Person>` subirebbe invece lo slicing: ogni elemento sarebbe soltanto una `Person`.

76 Distruttori virtuali

Slide 227–233.

Quando un oggetto derivato viene eliminato attraverso un puntatore alla classe base, il distruttore della base deve essere virtuale.

```
class Base {
public:
    virtual ~Base()
    {
        std::cout << "~Base\n";
    }
};

class Derived : public Base {
public:
    ~Derived() override
    {
        std::cout << "~Derived\n";
    }
};

// Dentro main():
Base* p = new Derived;
delete p;

// Ordine delle chiamate:
// ~Derived
// ~Base
```

Regola pratica: se una classe base deve permettere `delete` di oggetti derivati tramite un suo puntatore, darle un distruttore pubblico virtuale.

Se il distruttore della base non è virtuale, eliminare in questo modo un oggetto derivato causa **comportamento indefinito**. Non si tratta soltanto di una possibile perdita di memoria.

Il distruttore della derivata è automaticamente virtuale quando quello della base è virtuale.

Risorse possedute da una classe derivata

Nelle slide, `Student` possiede un puntatore a un vettore creato con `new`. Il suo distruttore deve liberarlo con `delete`.

```
// Frammenti della classe Student delle slide.
```

```
// Attributo:
std::vector<std::string>* courses_;

// Nel costruttore:
courses_ = new std::vector<std::string>();

// Per aggiungere un corso:
courses_->push_back("Physics");

// Nel distruttore:
delete courses_;
```

Se l'oggetto viene eliminato attraverso `Person*`, il distruttore virtuale permette di eseguire prima quello di `Student` e poi quello di `Person`.

Attenzione alle copie: la copia automatica di un puntatore proprietario copia l'indirizzo, non il vettore. Due oggetti potrebbero quindi tentare di eliminare la stessa risorsa. Occorre gestire correttamente copia e assegnazione oppure impedirle.

Quando non serve un'allocazione separata, si può conservare direttamente il vettore:

```
// Attributo alternativo:
std::vector<std::string> courses_;

// Aggiunta:
courses_.push_back("Physics");

// Il vettore libera automaticamente la sua memoria.
// Non occorrono new e delete per courses_.
```

77 Classi astratte e metodi virtuali puri

Slide 234–244.

Una classe astratta definisce un'interfaccia comune, lasciando alle classi derivate l'implementazione di alcune operazioni.

Un metodo virtuale puro si dichiara con `= 0`:

```
virtual double value(double x) const = 0;
```

`= 0` indica che il metodo è virtuale puro: non significa che restituisce zero.

Una classe con almeno un metodo virtuale puro ancora non implementato è astratta.

Esempio: interfaccia Function

```
#include <iostream>
#include <string>

class Function {
public:
    explicit Function(const std::string& name)
        : name_(name)
    {
    }

    virtual ~Function() = default;

    // Le derivate concrete devono implementarli.
    virtual double value(double x) const = 0;

    virtual double integrate(double a,
                             double b) const = 0;

    // Questi metodi hanno gia' un'implementazione.
    virtual std::string name() const
```

```
    {
        return name_;
    }

    virtual void print() const
    {
        std::cout << name_ << '\n';
    }

private:
    std::string name_;
};
```

Proprietà:

- Non si possono creare oggetti direttamente di tipo `Function`.
- Si possono usare puntatori e riferimenti a `Function` per accedere a oggetti di classi derivate concrete.
- Una classe astratta può avere attributi, costruttori e metodi già implementati. Il suo costruttore viene chiamato dalle derivate nella lista di inizializzazione.
- Una derivata che non implementa tutti i metodi virtuali puri rimane astratta.

```
// Function f("generica"); // Errore: classe astratta.

Function* p = nullptr; // Valido: e' un puntatore.
```

Precisazione: una classe astratta non è un tipo incompleto. Il compilatore conosce la sua struttura; è l'astrattezza a impedire di istanziarla.

Classe concreta: funzione costante

Per una funzione costante $f(x) = c$:

$$\int_a^b f(x)\,dx = c(b-a).$$

```
class ConstantFunction : public Function {
public:
    ConstantFunction(const std::string& name,
                     double value)
        : Function(name), value_(value)
    {
    }

    double value(double) const override
    {
        // Il valore non dipende da x.
        // Il parametro non usato non ha un nome.
        return value_;
    }

    double integrate(double a,
                     double b) const override
    {
        return value_ * (b - a);
    }

private:
    double value_;
};
```

```
// Dentro main():
ConstantFunction c("Costante", 2.0);

Function& f = c;
```

```
std::cout << f.value(10.0) << '\n';      // 2
std::cout << f.integrate(0.0, 3.0)
          << '\n';                       // 6

Function* p = new ConstantFunction("Altra", 4.0);
p->print();
delete p; // Corretto: ~Function() e' virtuale.
```

Perché evitare implementazioni fittizie

Scrivere nella base un metodo che restituisce sempre zero può nascondere un'implementazione dimenticata nella derivata.

```
// Implementazione fittizia: puo' nascondere errori.
virtual double integrate(double a,
                         double b) const
{
    return 0.0;
}
```

Se non esiste un comportamento generale sensato, si usa un metodo virtuale puro. Il compilatore impedisce di istanziare una derivata che non lo abbia implementato.

Esempio fisico: funzione gaussiana

Una gaussiana normalizzata con media μ e deviazione standard $\sigma > 0$ è:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2\right].$$

L'esempio seguente completa quello delle slide: usa la normalizzazione corretta e calcola anche l'integrale tramite `std::erf`:

$$\int_a^b f(x)\,dx = \frac{1}{2}\left[\operatorname{erf}\left(\frac{b-\mu}{\sqrt{2}\,\sigma}\right) - \operatorname{erf}\left(\frac{a-\mu}{\sqrt{2}\,\sigma}\right)\right].$$

```
#include <cmath>
#include <stdexcept>

class Gauss : public Function {
public:
    Gauss(const std::string& name,
          double mean,
          double sigma)
        : Function(name), mean_(mean), sigma_(sigma)
    {
        if (sigma <= 0.0) {
            throw std::invalid_argument(
                "sigma deve essere positiva");
        }
    }

    double value(double x) const override
    {
        const double pi = 3.141592653589793;
        const double pull = (x - mean_) / sigma_;

        return std::exp(-0.5 * pull * pull)
             / (sigma_ * std::sqrt(2.0 * pi));
    }

    double integrate(double a,
                     double b) const override
    {
        const double scale = std::sqrt(2.0) * sigma_;
```

```
        return 0.5 * (
            std::erf((b - mean_) / scale)
          - std::erf((a - mean_) / scale)
        );
    }

private:
    double mean_;
    double sigma_;
};
```

`throw std::invalid_argument` segnala un parametro non valido tramite un'eccezione. Per usare normalmente questo esempio, scegliere una `sigma` positiva.

`M_PI` non fa parte dello standard C++: definire la costante, come nell'esempio, evita problemi di portabilità.

78 Promemoria sul polimorfismo

- `virtual`: abilita la ridefinizione con scelta della versione in base al tipo dinamico.
- `override`: verifica che la firma ridefinisca correttamente un metodo virtuale.
- `= 0`: dichiara un metodo virtuale puro.
- `virtual`, `override` e `= 0`: solo nella dichiarazione dentro la classe.
- Classe astratta: non istanziabile, ma utilizzabile attraverso puntatori e riferimenti.
- Distruttore virtuale: permette la distruzione corretta tramite puntatore alla base.
- Copia per valore nella base: può causare object slicing.
- Nessuna chiamata virtuale nei costruttori e nei distruttori della base.
- Stampa polimorfica: `operator«` sulla base che chiama un `print(std::ostream&)` virtuale.
- Costruzione: prima la base, poi la derivata.
- Distruzione: prima la derivata, poi la base.

79 Funzioni matematiche e divisione intera

Integrazione dagli esami: buchi neri binari, frazioni, triangoli, binari compatte.

Funzione	Uso
`std::sqrt(x)`	Radice quadrata (`<cmath>`).
`std::pow(x, y)`	x^y, anche con esponente frazionario.
`std::exp, std::log`	Esponenziale e logaritmo naturale.
`std::sin, std::cos, std::tan`	Angoli in radianti.
`std::acos, std::asin`	Restituiscono radianti; argomento in $[-1, 1]$.
`std::atan2(y, x)`	Angolo di (x, y) in $(-\pi, \pi]$.
`std::abs(x)`	Valore assoluto (`<cmath>` per i double).
`std::erf(x)`	Funzione errore.
`std::max(a, b), std::min(a, b)`	Massimo e minimo (`<algorithm>`); `a` e `b` devono avere lo stesso tipo.

```
#include <algorithm>
#include <cmath>

const double pi = std::acos(-1.0); // Pi greco portabile.

double gammaAngle = std::acos((a*a + b*b - c*c) / (2.0*a*b));

double chiP = std::max(A1 * chi1Perp, A2 * chi2Perp);
// std::max(x, 2) con x double non compila: usa 2.0.
```

Divisione intera: l'errore più frequente

Se entrambi gli operandi sono interi, `/` esegue la divisione intera e tronca il risultato.

```
int a = 5 / 2;          // 2, non 2.5.
double b = 5 / 2;       // 2.0: la divisione e' gia' intera.
double c = 5.0 / 2;     // 2.5.

// Massa di chirp: (m1 m2)^(3/5) / (m1 + m2)^(1/5).
double sbagliato = std::pow(m1 * m2, 3 / 5);     // pow(...,0) = 1!
double corretto  = std::pow(m1 * m2, 3.0 / 5.0)
                 / std::pow(m1 + m2, 1.0 / 5.0);

double U = 0.5 * C * dV * dV; // Non 1/2 * C * dV * dV, che vale 0.
```

Per dividere due variabili intere come numeri reali:

```
int n = 3, d = 4;
double r = static_cast<double>(n) / d; // 0.75.
```

Regola pratica: nelle formule fisiche scrivi le costanti come `2.0`, `0.5`, `3.0 / 5.0`.

Resto e massimo comun divisore

`%` restituisce il resto della divisione intera. Il segno del resto segue quello del dividendo.

```
int r1 = 7 % 3;   // 1.
int r2 = -7 % 3;  // -1.
bool pari = (n % 2 == 0);
```

Massimo comun divisore con l'algoritmo di Euclide, utile per semplificare una frazione:

```
#include <cstdlib> // std::abs per gli int.

// Restituisce il massimo comun divisore, sempre >= 0.
int mcd(int a, int b)
{
    a = std::abs(a);
    b = std::abs(b);

    while (b != 0) {
        int resto = a % b;
        a = b;
        b = resto;
    }
    return a;
}
```

In C++17 esiste anche `std::gcd` in `<numeric>`. Per una frazione, dividi numeratore e denominatore per `mcd(n, d)` e scegli una convenzione per il segno, ad esempio denominatore sempre positivo. Il denominatore nullo va rifiutato nel costruttore e nei setter.

80 Validare i parametri: eccezioni e uscita

Integrazione dagli esami: intervalli fisici, denominatore nullo, dimensione dei vector.

Quando un esercizio chiede di *imporre* un vincolo (ad esempio $m_i \in [5, 100]\, M_\odot$ o $|\vec{\chi}| \leq 1$), il costruttore e i setter devono rifiutare i valori non validi.

```
#include <stdexcept>

// Funzione privata e static della classe:
// controlla un valore, usata da costruttore e setter.
void BinaryBH::checkMass(double m)
{
    if (m < 5.0 || m > 100.0) {
        throw std::out_of_range(
```

```
            "massa fuori da [5, 100] masse solari");
    }
}

BinaryBH::BinaryBH(double m1, double m2)
    : m1_(m1), m2_(m2)
{
    checkMass(m1);
    checkMass(m2);
}

void BinaryBH::setM1(double m1)
{
    checkMass(m1); // Stesso controllo anche nel setter.
    m1_ = m1;
}
```

Eccezione	Uso tipico
`std::invalid_argument`	Argomento non accettabile.
`std::out_of_range`	Valore fuori dall'intervallo ammesso.
`std::runtime_error`	Errore durante l'esecuzione, ad esempio un file non apribile.

Catturare l'eccezione nel main

```
#include <iostream>
#include <stdexcept>

int main()
{
    try {
        BinaryBH b(3.0, 20.0); // Massa non valida.
        std::cout << b << '\n';
    } catch (const std::exception& e) {
        std::cerr << "Errore: " << e.what() << '\n';
        return 1;
    }
    return 0;
}
```

- `throw` interrompe la funzione corrente e cerca un `catch` compatibile.
- `const std::exception&` cattura tutte le eccezioni standard; `e.what()` restituisce il messaggio.
- `std::cerr` è il flusso per i messaggi di errore.
- Se un costruttore lancia un'eccezione, l'oggetto non viene creato.

Uscire con un messaggio

Se il testo chiede esplicitamente di "uscire con un messaggio informativo":

```
#include <cstdlib>
#include <iostream>

if (field != "scalar") {
    std::cerr << "Campo non supportato: " << field << '\n';
    std::exit(EXIT_FAILURE);
}
```

`std::exit` termina subito il programma: gli oggetti locali non vengono distrutti. Quando possibile, un'eccezione è più flessibile, perché lascia decidere al chiamante.

Unità di misura: se usi unità particolari (ad esempio $M_\odot = 1$), scrivilo in un commento nell'header e nel messaggio stampato, come chiedono gli esami.

81 std::vector: dimensione e algoritmi

Integrazione dagli esami: polinomi, mesh, coefficienti.

```
#include <vector>

std::vector<double> a(5, 0.0);  // 5 elementi, tutti 0.0.
std::vector<double> b(5);       // 5 elementi inizializzati a 0.0.
std::vector<double> c{1.0, 2.0}; // Due elementi: 1.0 e 2.0.

a.resize(8, 0.0); // Ora 8 elementi; i nuovi valgono 0.0.
a.back();         // Ultimo elemento (vector non vuoto).
a.empty();        // true se non ci sono elementi.
```

Attenzione a tonde e graffe: `std::vector<double> v(3, 0.0)` ha tre elementi; `std::vector<double> v{3, 0.0}` ha due elementi, `3.0` e `0.0`.

Operazione	Effetto
`std::vector<T> v(n, x)`	`n` elementi uguali a `x`.
`v.resize(n, x)`	Cambia la dimensione; gli eventuali nuovi elementi valgono `x`.
`v.front(), v.back()`	Primo e ultimo elemento.
`v.empty()`	Vero se il vector è vuoto.
`v.clear()`	Rimuove tutti gli elementi.
`v = w`	Copia tutti gli elementi di `w`.

Controllare la dimensione ricevuta

```
Polynomial3D::Polynomial3D(const std::string& name,
                           const std::vector<double>& a)
    : Function(name), a_(a)
{
    if (a_.size() != 10) {
        throw std::invalid_argument("servono 10 coefficienti");
    }
}
```

Vector di vector e cicli annidati

```
// Matrice 3 x N: righe x, y, z.
std::vector<std::vector<double>> punti(
    3, std::vector<double>(N, 0.0));

punti[0][j] = x; // Coordinata x del punto j.

for (std::size_t i = 0; i < N; ++i) {
    for (std::size_t j = 0; j < N; ++j) {
        // Corpo eseguito N * N volte.
    }
}
```

Spesso è più chiaro un vector di `std::array<double, 3>` (sezione “Restituire più valori”).

82 Applicazione: classe Polynomial

Integrazione dagli esami: polinomi (3 esami).

Convenzione: `coeff[i]` è il coefficiente di x^i, quindi $p(x) = a_0 + a_1 x + a_2 x^2 + \dots$ e il grado è `coeff.size() - 1`.

Header: Polynomial.h

```
#ifndef POLYNOMIAL_H
#define POLYNOMIAL_H
```

```
#include <ostream>
#include <vector>

// Polinomio a coefficienti reali nella variabile x.
class Polynomial {
public:
    // coeff[i] e' il coefficiente di x^i.
    explicit Polynomial(const std::vector<double>& coeff);

    int degree() const;
    double value(double x) const;
    Polynomial derivative(int order = 1) const;
    double primitive(double x) const; // Costante nulla.
    double integral(double a, double b) const;

    Polynomial operator+(const Polynomial& rhs) const;
    Polynomial operator-(const Polynomial& rhs) const;
    Polynomial operator*(const Polynomial& rhs) const;
    Polynomial operator*(double k) const;

    const std::vector<double>& coefficients() const;

private:
    std::vector<double> coeff_;
};

Polynomial operator*(double k, const Polynomial& p);
std::ostream& operator<<(std::ostream& os,
                         const Polynomial& p);

#endif
```

Costruttore e valutazione (schema di Horner)

$$p(x) = a_0 + x\big(a_1 + x(a_2 + \cdots + x\,a_n)\big).$$

```
#include "Polynomial.h"
#include <algorithm>
#include <stdexcept>

Polynomial::Polynomial(const std::vector<double>& coeff)
    : coeff_(coeff)
{
    if (coeff_.empty()) {
        throw std::invalid_argument(
            "serve almeno un coefficiente");
    }
}

int Polynomial::degree() const
{
    return static_cast<int>(coeff_.size()) - 1;
}

double Polynomial::value(double x) const
{
    double result = 0.0;
    for (int i = degree(); i >= 0; --i) {
        result = result * x + coeff_[i];
    }
    return result;
}
```

L'indice del ciclo è un `int`: con `std::size_t`, la condizione `i >= 0` sarebbe sempre vera.

Derivata i-esima

$\frac{d}{dx}\sum_i a_i x^i = \sum_{i\geq 1} i\, a_i x^{i-1}$: il nuovo coefficiente di indice $i-1$ è $i\, a_i$.

```
Polynomial Polynomial::derivative(int order) const
{
    std::vector<double> c = coeff_;

    for (int k = 0; k < order; ++k) {
        if (c.size() == 1) { // Derivata di una costante.
            c[0] = 0.0;
            break;
        }
        std::vector<double> d(c.size() - 1);
        for (std::size_t i = 1; i < c.size(); ++i) {
            d[i - 1] = i * c[i];
        }
        c = d;
    }
    return Polynomial(c);
}
```

Primitiva e integrale definito

$P(x) = \sum_i \frac{a_i}{i+1} x^{i+1}, \qquad \int_a^b p(x)\,dx = P(b) - P(a).$

```
double Polynomial::primitive(double x) const
{
    double result = 0.0;
    double potenza = x; // x^(i+1).

    for (std::size_t i = 0; i < coeff_.size(); ++i) {
        result += coeff_[i] * potenza / (i + 1);
        potenza *= x;
    }
    return result;
}

double Polynomial::integral(double a, double b) const
{
    return primitive(b) - primitive(a);
}
```

Somma, differenza e prodotto

Nel prodotto, $a_i x^i \cdot b_j x^j$ contribuisce al coefficiente di indice $i+j$; il risultato ha `n + m - 1` coefficienti.

```
Polynomial Polynomial::operator+(const Polynomial& rhs) const
{
    std::size_t n = std::max(coeff_.size(),
                             rhs.coeff_.size());
    std::vector<double> c(n, 0.0);

    for (std::size_t i = 0; i < coeff_.size(); ++i) {
        c[i] += coeff_[i];
    }
    for (std::size_t i = 0; i < rhs.coeff_.size(); ++i) {
        c[i] += rhs.coeff_[i];
    }
    return Polynomial(c);
}

Polynomial Polynomial::operator-(const Polynomial& rhs) const
{
    return *this + rhs * (-1.0);
}
```

```
Polynomial Polynomial::operator*(const Polynomial& rhs) const
{
    std::vector<double> c(
        coeff_.size() + rhs.coeff_.size() - 1, 0.0);

    for (std::size_t i = 0; i < coeff_.size(); ++i) {
        for (std::size_t j = 0; j < rhs.coeff_.size(); ++j) {
            c[i + j] += coeff_[i] * rhs.coeff_[j];
        }
    }
    return Polynomial(c);
}

Polynomial Polynomial::operator*(double k) const
{
    std::vector<double> c = coeff_;
    for (double& ci : c) { // Riferimento: modifica c.
        ci *= k;
    }
    return Polynomial(c);
}

// Funzione libera: permette 2.0 * p.
Polynomial operator*(double k, const Polynomial& p)
{
    return p * k;
}
```

Per sommare un numero, basta costruire un polinomio costante: `p + Polynomial({3.0})`.

Stampa

```
std::ostream& operator<<(std::ostream& os,
                         const Polynomial& p)
{
    const std::vector<double>& c = p.coefficients();

    for (std::size_t i = 0; i < c.size(); ++i) {
        if (i > 0) {
            os << " + ";
        }
        os << c[i];
        if (i > 0) {
            os << " x^" << i;
        }
    }
    return os;
}
```

Attenzione: dopo una differenza possono restare coefficienti nulli in fondo, ad esempio $(x^2 + 1) - x^2$. Se il grado deve essere esatto, rimuovili con `while (c.size() > 1 && c.back() == 0.0) c.pop_back();`. Per `operator==` confronta i coefficienti con una tolleranza (sezione su `operator==`).

Una funzione razionale si può rappresentare con due attributi `Polynomial` (numeratore e denominatore): vedi la sezione sulla composizione.

83 Restituire più valori: struct e std::array

Integrazione dagli esami: gas ideale, campi vettoriali, spin.

struct per risultati con nomi

Una `struct` è una classe con membri `public` per default: adatta a raggruppare dati.

```cpp
// Risultato di una trasformazione termodinamica.
struct ProcessResult {
    double TE; // Temperatura finale [K].
    double Q;  // Calore scambiato [J].
    double W;  // Lavoro compiuto dal gas [J].
};

ProcessResult IdealGas::isobaric(double TS, double VS,
                                 double VE) const
{
    double TE = TS * VE / VS;
    double W = n_ * R * (TE - TS);
    double Q = n_ * cP() * (TE - TS);

    return {TE, Q, W}; // Stesso ordine dei membri.
}
```

```cpp
ProcessResult r = gas.isobaric(300.0, 1.0, 2.0);
std::cout << r.TE << ' ' << r.Q << ' ' << r.W << '\n';

auto [TE, Q, W] = gas.isobaric(300.0, 1.0, 2.0); // C++17.
```

Una costante come R si può definire nell'header, fuori dalle classi: `const double R = 8.314; // J/(mol K)`. A livello di namespace, una costante `const` può stare in un header incluso da più sorgenti.

std::array per vettori a dimensione fissa

```cpp
#include <array>

std::array<double, 3> chi1{0.1, 0.2, 0.3};

double chiPerp = std::sqrt(chi1[0] * chi1[0]
                         + chi1[1] * chi1[1]);

// Rotore di un campo vettoriale.
std::array<double, 3> VectorField::curl(
    double x, double y, double z) const
{
    return {
        fz_->deriv(x, y, z, 'y') - fy_->deriv(x, y, z, 'z'),
        fx_->deriv(x, y, z, 'z') - fz_->deriv(x, y, z, 'x'),
        fy_->deriv(x, y, z, 'x') - fx_->deriv(x, y, z, 'y')
    };
}
```

$$\nabla \cdot \vec{F} = \partial_x F_x + \partial_y F_y + \partial_z F_z, \qquad \nabla \times \vec{F} = \big(\partial_y F_z - \partial_z F_y,\ \partial_z F_x - \partial_x F_z,\ \partial_x F_y - \partial_y F_x\big).$$

Strumento	Quando usarlo
`struct`	Valori con significati diversi (T, Q, W).
`std::array<double, N>`	N componenti omogenee, N noto in compilazione (vettori 3D).
`std::vector<double>`	Numero di elementi variabile.
`std::pair`	Esattamente due valori.

84 Composizione: oggetti e puntatori come attributi

Integrazione dagli esami: filtri RLC, funzioni razionali, campi vettoriali.

Ereditarietà: "è un" (`Resistor` è un `CircuitElement`). Composizione: "ha un" (un filtro *ha* un resistore).

Oggetti come attributi

```
class RLCFilter {
public:
    RLCFilter(const Resistor& r, const Inductor& l,
              const Capacitor& c)
        : r_(r), l_(l), c_(c) // Obbligatorio: niente default.
    {
    }

    virtual ~RLCFilter() = default;

    // Comune a tutti i filtri.
    double omegaC() const
    {
        return 1.0 / std::sqrt(l_.L() * c_.C());
    }

    // Diverso per ogni topologia.
    virtual double bandwidth() const = 0;

    double quality() const
    {
        return omegaC() / bandwidth();
    }

    const Resistor& resistor() const { return r_; }

protected:
    Resistor r_;
    Inductor l_;
    Capacitor c_;
};
```

Se le classi degli attributi non hanno un costruttore di default, la lista di inizializzazione è obbligatoria. I getter possono restituire un riferimento const per evitare copie.

Allo stesso modo, una classe `Rational` può contenere due attributi `Polynomial`:

$$r(x) = \frac{N(x)}{D(x)}, \qquad r'(x) = \frac{N'(x)D(x) - N(x)D'(x)}{D(x)^2}.$$

Puntatori a una classe astratta come attributi

```
class VectorField {
public:
    VectorField(const Function* fx, const Function* fy,
                const Function* fz)
        : fx_(fx), fy_(fy), fz_(fz)
    {
        if (fx == nullptr || fy == nullptr
            || fz == nullptr) {
            throw std::invalid_argument("puntatore nullo");
        }
    }

    double divergence(double x, double y, double z) const
    {
        return fx_->deriv(x, y, z, 'x')
             + fy_->deriv(x, y, z, 'y')
             + fz_->deriv(x, y, z, 'z');
    }

private:
    const Function* fx_; // Non posseduti:
    const Function* fy_; // niente delete
    const Function* fz_; // nel distruttore.
};
```

```
// Dentro main():
Polynomial3D fx("Fx", {0, 0, 1, 0, 0, 0, 0, 0, 0, 0}); // y
Polynomial3D fy("Fy", {0, -1, 0, 0, 0, 0, 0, 0, 0, 0}); // -x
Polynomial3D fz("Fz", {0, 0, 0, 0, 0, 0, 1, 0, 0, 0}); // z^2

VectorField F(&fx, &fy, &fz);
std::cout << F.divergence(1.0, 2.0, 3.0) << '\n'; // 6
```

Attenzione: gli oggetti puntati devono esistere finché si usa il `VectorField`. Una classe astratta non può essere un attributo per valore: servono puntatori o riferimenti.

Se il testo fornisce una classe base senza distruttore virtuale, aggiungi `virtual ~Function() = default;` e segnalalo in un commento.

85 Contatori per classe e operatori su elementi

Integrazione dagli esami: circuiti RLC, condensatori.

Un contatore per ogni classe derivata

Se l'utente deve conoscere il numero di resistori, induttori e condensatori separatamente, ogni classe concreta ha il proprio attributo static.

```
// Resistor.h
class Resistor : public CircuitElement {
public:
    explicit Resistor(double R);
    Resistor(const Resistor& other);
    ~Resistor() override;

    static int count();

    Resistor operator+(const Resistor& rhs) const;  // Serie.
    Resistor operator||(const Resistor& rhs) const; // Parallelo.

private:
    double R_;
    static int count_;
};
```

```
// Resistor.cc
int Resistor::count_ = 0;

Resistor::Resistor(double R)
    : CircuitElement("R"), R_(R)
{
    ++count_;
}

Resistor::Resistor(const Resistor& other)
    : CircuitElement(other), R_(other.R_)
{
    ++count_;
}

Resistor::~Resistor()
{
    --count_;
}

int Resistor::count()
{
    return count_;
}
```

Attenzione: in un costruttore di copia scritto a mano, chiama anche il costruttore di copia della base nella lista di inizializzazione, come nell'esempio. Altrimenti la parte base viene costruita con il costruttore di default, se esiste, e non viene copiata.

Serie e parallelo con + e ||

$$R_s = R_1 + R_2, \qquad R_p = \left(\frac{1}{R_1} + \frac{1}{R_2}\right)^{-1}.$$

```
Resistor Resistor::operator+(const Resistor& rhs) const
{
    return Resistor(R_ + rhs.R_);
}

Resistor Resistor::operator||(const Resistor& rhs) const
{
    return Resistor(1.0 / (1.0 / R_ + 1.0 / rhs.R_));
}
```

```
Resistor r1(100.0), r2(100.0);
Resistor serie = r1 + r2;    // 200 ohm.
Resistor par = r1 || r2;     // 50 ohm.
Resistor misto = (r1 + r2) || r1; // Parentesi esplicite.
```

Per gli induttori vale la stessa forma; per i condensatori le formule si scambiano: in parallelo si sommano, in serie si usa l'inverso.

Attenzione: `||` ha precedenza più bassa di `+`: `r1 + r2 || r3` significa `(r1 + r2) || r3`. Usa sempre le parentesi. Un `operator||` sovraccaricato valuta sempre entrambi gli operandi.

86 Named constructor in una classe derivata

Integrazione dagli esami: triangolo rettangolo.

```
class RightAngleTriangle : public Triangle {
public:
    // Dai due cateti.
    static RightAngleTriangle fromLegs(
        const std::string& label, double a, double b)
    {
        return RightAngleTriangle(
            label, a, b, std::sqrt(a * a + b * b));
    }

    // Dall'ipotenusa c e da un cateto a.
    static RightAngleTriangle fromHypotenuseAndLeg(
        const std::string& label, double c, double a)
    {
        if (c <= a) {
            throw std::invalid_argument(
                "l'ipotenusa deve superare il cateto");
        }
        return RightAngleTriangle(
            label, a, std::sqrt(c * c - a * a), c);
    }

private:
    // Privato: si crea solo tramite le funzioni sopra.
    RightAngleTriangle(const std::string& label,
                       double a, double b, double c)
        : Triangle(label, a, b, c)
    {
    }
};
```

```
RightAngleTriangle t =
    RightAngleTriangle::fromLegs("t1", 3.0, 4.0);
```

Le funzioni factory calcolano il lato mancante e poi chiamano il costruttore della base tramite quello privato della derivata.

87 Controllo del flusso: while, else if, switch

Integrazione dagli esami.

while, else if e operatori logici

```
while (b != 0) {      // Controllo prima di ogni giro.
    // ...
}

if (x < 0.0) {
    // ...
} else if (x == 0.0) {
    // ...
} else {
    // ...
}
```

Operatore	Significato	Esempio
`&&`	AND logico	`a > 0 && b > 0`
`||`	OR logico	`m < 5.0 || m > 100.0`
`!`	NOT logico	`!valido`
`>`, `<=`	Confronti	`x <= 1.0`

switch su un carattere

```
double Polynomial3D::deriv(double x, double y, double z,
                           char var) const
{
    switch (var) {
        case 'x':
            return a_[1] + 2.0 * a_[4] * x
                 + a_[7] * y + a_[8] * z;
        case 'y':
            return a_[2] + 2.0 * a_[5] * y
                 + a_[7] * x + a_[9] * z;
        case 'z':
            return a_[3] + 2.0 * a_[6] * z
                 + a_[8] * x + a_[9] * y;
        default:
            throw std::invalid_argument(
                "var deve essere 'x', 'y' o 'z'");
    }
}
```

Qui ogni `case` termina con `return`. Se un `case` non esce dalla funzione, serve `break`: senza di esso l'esecuzione prosegue nel `case` successivo. `switch` funziona con interi e caratteri, non con `std::string`.

Caratteri e stringhe

```
char var = 'x';             // Apici singoli: un carattere.
std::string field = "scalar"; // Virgolette: una stringa.

if (field == "scalar") {    // Confronto tra stringhe.
```

```
    // ...
}

std::string nome = "Mesh " + std::to_string(N);
```

88 File di testo

Integrazione dagli esami: scrittura del potenziale su file.

Scrivere

```
#include <fstream>
#include <string>

// Restituisce true se la scrittura riesce.
bool Voltage::writeToFile(const std::string& filename) const
{
    std::ofstream out(filename);
    if (!out) {
        return false; // File non apribile.
    }

    out << "# x y z V\n";
    for (std::size_t j = 0; j < V_.size(); ++j) {
        out << x_[j] << ' ' << y_[j] << ' '
            << z_[j] << ' ' << V_[j] << '\n';
    }
    return true;
} // Il file si chiude automaticamente qui.
```

`std::ofstream` si usa come `std::cout`; funzionano anche `operator«` e i manipolatori come `std::setprecision`. Restituire `bool` evita un `void` che nasconderebbe un errore.

Leggere

```
std::ifstream in("dati.txt");
if (!in) {
    throw std::runtime_error("impossibile aprire dati.txt");
}

double x = 0.0, y = 0.0;
while (in >> x >> y) { // Si ferma a fine file o errore.
    // Usa x e y.
}
```

89 Numeri casuali

Integrazione dagli esami: mesh casuale, parametri estratti.

```
#include <random>

std::mt19937 gen(12345); // Generatore con seme fisso.

std::uniform_real_distribution<double> unif(-L / 2.0,
                                            L / 2.0);
std::normal_distribution<double> gauss(0.0, 1.0);

double x = unif(gen); // Uniforme in [-L/2, L/2).
double g = gauss(gen); // Media 0, deviazione 1.
```

Attenzione: crea il generatore una sola volta e riusalo. Ricrearlo con lo stesso seme a ogni chiamata produce sempre gli stessi numeri. Dentro una funzione si può usare `static std::mt19937 gen(12345);`, che viene inizializzato una volta sola. Per un seme diverso a ogni esecuzione: `std::mt19937 gen(std::random_device{}());`.

Direzione casuale uniforme sulla sfera

Estrarre θ uniforme in $[0, \pi]$ è sbagliato: le direzioni si addensano ai poli. Si estrae invece $\cos\theta$ uniforme in $[-1, 1]$.

```
std::uniform_real_distribution<double> u(0.0, 1.0);
const double pi = std::acos(-1.0);

double cosTheta = 2.0 * u(gen) - 1.0;
double phi = 2.0 * pi * u(gen);
double sinTheta = std::sqrt(1.0 - cosTheta * cosTheta);

double modulo = u(gen); // |chi| uniforme in [0, 1).
std::array<double, 3> chi{
    modulo * sinTheta * std::cos(phi),
    modulo * sinTheta * std::sin(phi),
    modulo * cosTheta
};
```

90 Algoritmi utili sui vector

Integrazione dagli esami: DataSet, mediana, poligoni, cariche puntiformi.

```
#include <algorithm> // swap, sort, min_element, max_element
#include <numeric>   // accumulate
#include <vector>

std::vector<double> v{1.0, 2.0, 3.0, 4.0};

std::swap(v[0], v[3]);          // Scambia due elementi.
v.erase(v.begin() + 1);         // Rimuove l'elemento di indice 1.
v.insert(v.begin() + 1, 9.0);   // Inserisce 9.0 in posizione 1.

std::sort(v.begin(), v.end());  // Ordine crescente.

double minimo = *std::min_element(v.begin(), v.end());
double massimo = *std::max_element(v.begin(), v.end());
double somma = std::accumulate(v.begin(), v.end(), 0.0);
```

- `v.begin() + i` è un iteratore all'elemento `i`: controlla prima che `i < v.size()`.
- `min_element` e `max_element` restituiscono un iteratore: `*` ne legge il valore. Il vector non deve essere vuoto.
- In `std::accumulate` scrivi `0.0`, non `0`: con `0` la somma verrebbe fatta tra interi.
- Per scambiare due punti salvati come oggetti, `std::swap(points_[i], points_[j])` scambia gli oggetti interi.

Mediana

```
// Parametro per valore: ordina una copia
// senza modificare i dati originali.
double median(std::vector<double> v)
{
    if (v.empty()) {
        throw std::invalid_argument("nessun dato");
    }
    std::sort(v.begin(), v.end());

    std::size_t n = v.size();
    if (n % 2 == 1) {
```

```
        return v[n / 2];
    }
    return 0.5 * (v[n / 2 - 1] + v[n / 2]);
}
```

Dentro un metodo `const` non puoi ordinare l'attributo: passa `x_` a questa funzione, che lavora su una copia.

Altre medie

$$G = \Big(\prod_{i=1}^{N} x_i\Big)^{1/N} = \exp\Big(\frac{1}{N}\sum_{i=1}^{N} \ln x_i\Big), \qquad H = \frac{N}{\sum_{i=1}^{N} x_i^{-1}}.$$

```
double geometricMean(const std::vector<double>& x)
{
    // Somma dei logaritmi: evita overflow del prodotto.
    double sommaLog = 0.0;
    for (double xi : x) {
        sommaLog += std::log(xi); // Richiede xi > 0.
    }
    return std::exp(sommaLog / x.size());
}
```

Indici circolari

Per il perimetro di un poligono, l'ultimo vertice si collega al primo:

```
double perimetro = 0.0;
for (std::size_t i = 0; i < n; ++i) {
    std::size_t j = (i + 1) % n; // Dopo n-1 torna a 0.
    double dx = x[j] - x[i];
    double dy = y[j] - y[i];
    perimetro += std::sqrt(dx * dx + dy * dy);
}
```

91 Applicazione: classe Matrix

Integrazione dagli esami: matrici reali.

Header: Matrix.h

```
#ifndef MATRIX_H
#define MATRIX_H

#include <ostream>
#include <vector>

// Matrice reale nRows x nCols.
class Matrix {
public:
    Matrix();                      // Matrice vuota 0 x 0.
    explicit Matrix(int n);        // Quadrata n x n, zeri.
    Matrix(int nRows, int nCols); // Rettangolare, zeri.
    explicit Matrix(const std::vector<double>& diag);
    explicit Matrix(const std::vector<std::vector<double>>& els);
    ~Matrix() = default; // vector libera da solo la memoria.

    int nRows() const;
    int nCols() const;

    double element(int i, int j) const;
    void setElement(int i, int j, double value);
    std::vector<double> row(int i) const;
```

```
    std::vector<double> column(int j) const;

    void swapRows(int i, int k);
    void swapColumns(int j, int l);
    Matrix transpose() const;

    Matrix operator+(const Matrix& rhs) const;
    Matrix operator-(const Matrix& rhs) const;
    Matrix operator*(const Matrix& rhs) const;

    void print(std::ostream& os) const;

private:
    void checkIndices(int i, int j) const;

    int nRows_;
    int nCols_;
    std::vector<std::vector<double>> els_; // els_[riga][colonna]
};

#endif
```

Costruttori

```
#include "Matrix.h"
#include <stdexcept>
#include <utility>

Matrix::Matrix()
    : nRows_(0), nCols_(0)
{
}

Matrix::Matrix(int n)
    : Matrix(n, n) // Delega all'altro costruttore.
{
}

Matrix::Matrix(int nRows, int nCols)
    : nRows_(nRows), nCols_(nCols),
      els_(nRows, std::vector<double>(nCols, 0.0))
{
    if (nRows <= 0 || nCols <= 0) {
        throw std::invalid_argument("dimensioni non positive");
    }
}

Matrix::Matrix(const std::vector<double>& diag)
    : Matrix(static_cast<int>(diag.size()))
{
    for (int i = 0; i < nRows_; ++i) {
        els_[i][i] = diag[i];
    }
}

Matrix::Matrix(const std::vector<std::vector<double>>& els)
    : nRows_(static_cast<int>(els.size())),
      nCols_(els.empty() ? 0 : static_cast<int>(els[0].size())),
      els_(els)
{
    for (const auto& r : els_) {
        if (static_cast<int>(r.size()) != nCols_) {
            throw std::invalid_argument(
                "righe di lunghezza diversa");
        }
    }
```

```
}
```

`Matrix(n) : Matrix(n, n)` è un **costruttore delegante** (C++11): riusa un altro costruttore della stessa classe nella lista di inizializzazione. `condizione ? a : b` vale `a` se la condizione è vera, altrimenti `b`.

Accesso, scambi e trasposta

```
void Matrix::checkIndices(int i, int j) const
{
    if (i < 0 || i >= nRows_ || j < 0 || j >= nCols_) {
        throw std::out_of_range("indice fuori dalla matrice");
    }
}

double Matrix::element(int i, int j) const
{
    checkIndices(i, j);
    return els_[i][j];
}

std::vector<double> Matrix::column(int j) const
{
    checkIndices(0, j);
    std::vector<double> c(nRows_);
    for (int i = 0; i < nRows_; ++i) {
        c[i] = els_[i][j];
    }
    return c;
}

void Matrix::swapRows(int i, int k)
{
    checkIndices(i, 0);
    checkIndices(k, 0);
    std::swap(els_[i], els_[k]); // Scambia le righe intere.
}

void Matrix::swapColumns(int j, int l)
{
    checkIndices(0, j);
    checkIndices(0, l);
    for (int i = 0; i < nRows_; ++i) {
        std::swap(els_[i][j], els_[i][l]);
    }
}

Matrix Matrix::transpose() const
{
    Matrix t(nCols_, nRows_);
    for (int i = 0; i < nRows_; ++i) {
        for (int j = 0; j < nCols_; ++j) {
            t.els_[j][i] = els_[i][j];
        }
    }
    return t;
}
```

Un metodo della classe può accedere agli attributi privati di un altro oggetto della stessa classe, come `t.els_`.

Somma e prodotto

$$(AB)_{ij} = \sum_k A_{ik} B_{kj}, \qquad A : n \times m,\ B : m \times p \ \Rightarrow\ AB : n \times p.$$

```
Matrix Matrix::operator+(const Matrix& rhs) const
{
```

```
    if (nRows_ != rhs.nRows_ || nCols_ != rhs.nCols_) {
        throw std::invalid_argument("dimensioni diverse");
    }
    Matrix s(nRows_, nCols_);
    for (int i = 0; i < nRows_; ++i) {
        for (int j = 0; j < nCols_; ++j) {
            s.els_[i][j] = els_[i][j] + rhs.els_[i][j];
        }
    }
    return s;
}

Matrix Matrix::operator*(const Matrix& rhs) const
{
    if (nCols_ != rhs.nRows_) {
        throw std::invalid_argument(
            "colonne di A diverse dalle righe di B");
    }
    Matrix p(nRows_, rhs.nCols_); // Inizializzata a zero.
    for (int i = 0; i < nRows_; ++i) {
        for (int j = 0; j < rhs.nCols_; ++j) {
            for (int k = 0; k < nCols_; ++k) {
                p.els_[i][j] += els_[i][k] * rhs.els_[k][j];
            }
        }
    }
    return p;
}

void Matrix::print(std::ostream& os) const
{
    for (const auto& r : els_) {
        for (double x : r) {
            os << x << '\t';
        }
        os << '\n';
    }
}
```

La differenza è identica alla somma, con `-`. Il prodotto di matrici non è commutativo: `A * B` e `B * A` possono anche non essere definiti entrambi.

92 enum class: un valore tra pochi casi

Integrazione dagli esami: coniche, capacità date da Q, ΔV o U, scelta di distribuzioni.

```
enum class ConicType { Ellipse, Parabola, Hyperbola };

ConicType t = ConicType::Ellipse;  // Nome qualificato.

if (t == ConicType::Parabola) {
    // ...
}
```

Un `enum class` definisce un tipo con un insieme fisso di valori dal nome esplicito. È più chiaro e sicuro di un `int` o di una stringa per rappresentare "ellisse, parabola o iperbole".

Classificare e stampare

```
// Classificazione tramite il discriminante b^2 - 4ac.
ConicType classify(double a, double b, double c)
{
    const double tol = 1e-12;
    double delta = b * b - 4.0 * a * c;
```

```
    if (std::abs(delta) < tol) {
        return ConicType::Parabola; // Delta circa zero.
    }
    if (delta < 0.0) {
        return ConicType::Ellipse;
    }
    return ConicType::Hyperbola;
}

std::string toString(ConicType t)
{
    switch (t) {
        case ConicType::Ellipse:   return "ellisse";
        case ConicType::Parabola:  return "parabola";
        case ConicType::Hyperbola: return "iperbole";
    }
    return "sconosciuto"; // Evita l'avviso di -Wall.
}
```

Un attributo `ConicType type_;` si aggiorna nel costruttore e in ogni setter dei coefficienti. `operator«` può stampare `toString(type_)`.

enum come "etichetta" nel costruttore

Alternativa alle funzioni factory quando tre costruttori avrebbero gli stessi tipi (carica, tensione o energia):

```
enum class Given { Charge, Voltage, Energy };

Capacitor::Capacitor(double R1, double R2, double epsR,
                     Given what, double value)
    : R1_(R1), R2_(R2), epsR_(epsR)
{
    const double pi = std::acos(-1.0);
    const double eps0 = 8.854e-12; // [F/m]
    C_ = 4.0 * pi * eps0 * epsR_ * R1_ * R2_ / (R2_ - R1_);

    switch (what) {
        case Given::Charge:  Q_ = value;                      break;
        case Given::Voltage: Q_ = C_ * value;                 break;
        case Given::Energy:  Q_ = std::sqrt(2.0 * C_ * value); break;
    }
}
```

```
Capacitor c(0.01, 0.02, 1.0, Given::Voltage, 100.0);
```

Da $U = Q^2/(2C)$ segue $Q = \sqrt{2CU}$. Qui C si calcola nel costruttore della classe che conosce la geometria: nessuna chiamata virtuale.

93 Scegliere la classe derivata a runtime: std::unique_ptr

Integrazione dagli esami: metodo di integrazione letto da file.

Quando il tipo dell'oggetto dipende da un input (ad esempio `"Euler"` oppure `"RungeKutta2"`), si crea la derivata e la si usa tramite la base.

```
#include <memory>
#include <stdexcept>
#include <string>

std::unique_ptr<Integrator> makeIntegrator(
    const std::string& method, double dt)
{
    if (method == "Euler") {
        return std::make_unique<Euler>(dt);
    }
```

```
    if (method == "RungeKutta2") {
        return std::make_unique<RungeKutta2>(dt);
    }
    throw std::invalid_argument("metodo sconosciuto: " + method);
}
```

```
// Dentro main():
std::unique_ptr<Integrator> integ =
    makeIntegrator(method, dt);

integ->simulate(orbit, u0, nSteps); // Chiamata virtuale.
// Nessun delete: unique_ptr libera l'oggetto da solo.
```

- `std::unique_ptr<T>` possiede l'oggetto puntato e lo distrugge quando esce dall'ambito.
- `std::make_unique<D>(args)` crea un `D` (C++14 o successivo).
- Si usa con `->` come un puntatore normale.
- Non si può copiare: un solo proprietario alla volta.
- La base deve comunque avere un distruttore virtuale.

```
// Collezione di oggetti polimorfici posseduti dal vector.
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Rectangle>("r", 2.0, 3.0));

for (const auto& s : shapes) {
    std::cout << *s << '\n';
}
```

Senza `unique_ptr` si può usare `new` e `delete` con un `Integrator*`: in quel caso ricordati il `delete` alla fine.

94 Integrazione numerica: Euler e Runge-Kutta 2

Integrazione dagli esami: moto di un satellite.

Un sistema di equazioni del second'ordine si riscrive al primo ordine con lo stato $u = (x, y, z, v_x, v_y, v_z)$ e $du/dt = f(t, u)$.

$$\text{Euler: } u_{i+1} = u_i + f(t_i, u_i)\,\Delta t,$$
$$\text{RK2: } K_1 = f(t_i, u_i), \quad K_2 = f\big(t_i + \tfrac{\Delta t}{2},\, u_i + K_1 \tfrac{\Delta t}{2}\big), \quad u_{i+1} = u_i + K_2\,\Delta t.$$

Il sistema fisico

```
#include <array>
#include <cmath>

using State = std::array<double, 6>; // x, y, z, vx, vy, vz

// Sistema descritto da du/dt = f(t, u).
class System {
public:
    virtual ~System() = default;
    virtual State derivative(double t, const State& u) const = 0;
};

// Moto in un campo gravitazionale centrale.
class Orbit : public System {
public:
    Orbit(double G, double M) : G_(G), M_(M) {}

    State derivative(double, const State& u) const override
```

```
    {
        double r = std::sqrt(u[0] * u[0] + u[1] * u[1]
                           + u[2] * u[2]);
        double k = -G_ * M_ / (r * r * r);

        // dr/dt = v; dv/dt = -GM r / |r|^3.
        return {u[3], u[4], u[5],
                k * u[0], k * u[1], k * u[2]};
    }

private:
    double G_;
    double M_;
};
```

La forza di attrito si aggiunge alle tre componenti dell'accelerazione, divisa per la massa del satellite.

Gli integratori

```
#include <vector>

// Restituisce u + h * k, componente per componente.
State addScaled(const State& u, const State& k, double h)
{
    State r;
    for (std::size_t i = 0; i < u.size(); ++i) {
        r[i] = u[i] + h * k[i];
    }
    return r;
}

class Integrator {
public:
    explicit Integrator(double dt) : dt_(dt) {}
    virtual ~Integrator() = default;

    // Un passo: diverso per ogni metodo.
    virtual State step(const System& sys, double t,
                       const State& u) const = 0;

    // Ciclo comune: scritto una volta nella base.
    std::vector<State> simulate(const System& sys,
                                const State& u0,
                                int nSteps) const
    {
        std::vector<State> traj{u0};
        State u = u0;
        double t = 0.0;
        for (int i = 0; i < nSteps; ++i) {
            u = step(sys, t, u); // Chiamata virtuale.
            t += dt_;
            traj.push_back(u);
        }
        return traj;
    }

protected:
    double dt_;
};

class Euler : public Integrator {
public:
    explicit Euler(double dt) : Integrator(dt) {}

    State step(const System& sys, double t,
               const State& u) const override
```

```
    {
        return addScaled(u, sys.derivative(t, u), dt_);
    }
};

class RungeKutta2 : public Integrator {
public:
    explicit RungeKutta2(double dt) : Integrator(dt) {}

    State step(const System& sys, double t,
               const State& u) const override
    {
        State K1 = sys.derivative(t, u);
        State K2 = sys.derivative(
            t + 0.5 * dt_, addScaled(u, K1, 0.5 * dt_));
        return addScaled(u, K2, dt_);
    }
};
```

Verifica rapida: prova gli integratori su un oscillatore armonico ($\dot{x} = v$, $\dot{v} = -x$): dopo un periodo RK2 torna molto più vicino al punto iniziale di Euler.

Scrivere la traiettoria

```
std::ofstream out("sim.dat");
out << "# x y z vx vy vz\n";
for (const State& u : traj) {
    for (double ui : u) {
        out << ui << ' ';
    }
    out << '\n';
}
```

95 Leggere un file di parametri

Integrazione dagli esami: params.ini, cariche da file.

Formato di esempio, una coppia "nome valore" per riga:

```
# Parametri della simulazione
method RungeKutta2
nSteps 10000
dt 1.0
```

```
#include <fstream>
#include <map>
#include <sstream>
#include <stdexcept>
#include <string>

std::map<std::string, std::string> readParams(
    const std::string& filename)
{
    std::ifstream in(filename);
    if (!in) {
        throw std::runtime_error("impossibile aprire " + filename);
    }

    std::map<std::string, std::string> params;
    std::string line;

    while (std::getline(in, line)) {            // Una riga alla volta.
        if (line.empty() || line[0] == '#') {
            continue;                            // Salta vuote e commenti.
```

```
        }
        std::istringstream iss(line);            // Legge dalla riga.
        std::string key, value;
        if (iss >> key >> value) {
            params[key] = value;
        }
    }
    return params;
}
```

```
auto params = readParams("params.ini");

std::string method = params.at("method");
int nSteps = std::stoi(params.at("nSteps")); // Testo -> int.
double dt = std::stod(params.at("dt"));      // Testo -> double.
```

- `std::getline(in, line)` legge una riga intera.
- `std::istringstream` permette di usare » su una stringa, come su `std::cin`.
- `continue` passa subito al giro successivo del ciclo.
- `params.at(...)` genera un'eccezione se il parametro manca nel file.

Leggere oggetti da file

Se ogni riga contiene "carica x y z":

```
std::ifstream in(filename);
double q = 0.0, x = 0.0, y = 0.0, z = 0.0;

// Supponendo PointCharge(double q, std::array<double, 3> r).
while (in >> q >> x >> y >> z) {
    charges_.push_back(PointCharge(q, {x, y, z}));
}
```

96 Altri numeri casuali

Integrazione dagli esami: tavola di Galton, ensemble di particelle.

```
#include <random>

std::mt19937 gen(12345);

std::bernoulli_distribution destra(0.5);        // true con p = 0.5.
std::uniform_int_distribution<int> dado(1, 6);   // Interi in [1, 6].

if (destra(gen)) {
    ++k; // La pallina va a destra.
}
int faccia = dado(gen);
```

`uniform_int_distribution` include entrambi gli estremi; `uniform_real_distribution` esclude quello superiore.

Array triangolare

Per una tavola con righe $r = 0, \ldots, N-1$, dove la riga r ha $r+1$ caselle, la casella k della riga r ha indice:

```
std::size_t index(std::size_t r, std::size_t k)
{
    return r * (r + 1) / 2 + k; // Prime r righe + posizione.
}

std::vector<int> board(N * (N + 1) / 2, 0); // Tutte a zero.
```

Qui la divisione intera è corretta: $r(r+1)$ è sempre pari.

97 Scheletro d'esame

Struttura minima da adattare: base astratta, derivata concreta, contatore, stampa, validazione.

Shape.h

```
#ifndef SHAPE_H
#define SHAPE_H

#include <ostream>
#include <string>

// Classe base astratta per tutte le forme.
class Shape {
public:
    explicit Shape(const std::string& label);
    Shape(const Shape& other);
    virtual ~Shape();

    // Ogni classe concreta deve definire la stampa.
    virtual void print(std::ostream& os) const = 0;

    const std::string& label() const;
    void setLabel(const std::string& label);

    // Numero di forme attualmente esistenti.
    static int count();

private:
    std::string label_;
    static int count_;
};

// Stampa polimorfica: chiama print() della derivata.
std::ostream& operator<<(std::ostream& os, const Shape& s);

#endif
```

Shape.cc

```
#include "Shape.h"

int Shape::count_ = 0; // Definizione unica dello static.

Shape::Shape(const std::string& label)
    : label_(label)
{
    ++count_;
}

Shape::Shape(const Shape& other)
    : label_(other.label_)
{
    ++count_; // Anche le copie sono nuove istanze.
}

Shape::~Shape()
{
    --count_;
}
```

```
const std::string& Shape::label() const
{
    return label_;
}

void Shape::setLabel(const std::string& label)
{
    label_ = label;
}

int Shape::count()
{
    return count_;
}

std::ostream& operator<<(std::ostream& os, const Shape& s)
{
    s.print(os); // Chiamata virtuale.
    return os;
}
```

Rectangle.h

```
#ifndef RECTANGLE_H
#define RECTANGLE_H

#include "Shape.h"

// Rettangolo di lati a e b [m].
class Rectangle : public Shape {
public:
    Rectangle(const std::string& label, double a, double b);

    double a() const;
    double b() const;
    double area() const;      // [m^2]
    double perimeter() const; // [m]

    void setSides(double a, double b);

    void print(std::ostream& os) const override;

private:
    // Ricalcola area e perimetro. Non e' virtuale:
    // si puo' chiamare nel costruttore.
    void update();

    double a_;
    double b_;
    double area_;
    double perimeter_;
};

#endif
```

Rectangle.cc

```
#include "Rectangle.h"
#include <stdexcept>

Rectangle::Rectangle(const std::string& label,
                     double a, double b)
    : Shape(label), a_(a), b_(b)
{
```

```
    if (a <= 0.0 || b <= 0.0) {
        throw std::invalid_argument(
            "i lati devono essere positivi");
    }
    update();
}

double Rectangle::a() const { return a_; }
double Rectangle::b() const { return b_; }
double Rectangle::area() const { return area_; }
double Rectangle::perimeter() const { return perimeter_; }

void Rectangle::setSides(double a, double b)
{
    if (a <= 0.0 || b <= 0.0) {
        throw std::invalid_argument(
            "i lati devono essere positivi");
    }
    a_ = a;
    b_ = b;
    update(); // I valori memorizzati restano coerenti.
}

void Rectangle::print(std::ostream& os) const
{
    os << "Rectangle " << label() << ": a = " << a_
       << ", b = " << b_ << ", area = " << area_;
}

void Rectangle::update()
{
    area_ = a_ * b_;
    perimeter_ = 2.0 * (a_ + b_);
}
```

Valori calcolati e memorizzati: quando il testo chiede di "calcolare e memorizzare" (area, volume, angoli), ricalcolali in un unico metodo privato chiamato dal costruttore e da *ogni* setter. Un metodo come `dilate` modifica le dimensioni tramite gli stessi setter, così i valori restano coerenti.

app.cpp

```
#include <iostream>
#include <stdexcept>
#include <vector>

#include "Rectangle.h"

int main()
{
    try {
        Rectangle r("r1", 2.0, 3.0);
        std::cout << r << '\n';

        r.setSides(4.0, 5.0);
        std::cout << r << '\n';

        // Uso polimorfico tramite la base.
        std::vector<const Shape*> shapes{&r};
        for (const Shape* s : shapes) {
            std::cout << *s << '\n';
        }

        std::cout << "Forme esistenti: "
                  << Shape::count() << '\n';

        Rectangle bad("r2", -1.0, 2.0); // Eccezione.
```

```
    } catch (const std::exception& e) {
        std::cerr << "Errore: " << e.what() << '\n';
        return 1;
    }

    return 0;
}
```

Compilazione

```
clang++ -std=c++17 -Wall -c Shape.cc
clang++ -std=c++17 -Wall -c Rectangle.cc
clang++ -std=c++17 -Wall app.cpp Shape.o Rectangle.o -o app
./app
```

98 Checklist prima di consegnare

Ricavata dai criteri di valutazione degli esami.

- **Separazione:** ogni classe ha `.h` con include guards e `.cc`; c'è un `main` o `app.cpp` che mostra l'uso delle classi.
- **Tipi di ritorno:** un metodo che calcola qualcosa *restituisce* il risultato; niente `void` che stampa invece di restituire.
- **Argomenti:** tipi semplici per valore, oggetti, stringhe e vector per `const&`.
- **Getter e setter:** getter `const`; setter che validano e aggiornano i valori memorizzati.
- **Gerarchia:** distruttore virtuale nella base, `= 0` dove il testo dice "must require", `override` nelle derivate, nessun `virtual` nei file `.cc`.
- **Costruttori:** lista di inizializzazione; nessuna chiamata virtuale nel costruttore della base; costruttore di copia se richiesto o se c'è un contatore.
- **Static:** definizione nel `.cc`; contatore aggiornato in tutti i costruttori e nel distruttore.
- **Formule:** `0.5`, `3.0 / 5.0`; angoli in radianti; costanti con nome e unità.
- **Validazione:** vincoli fisici controllati nel costruttore e nei setter; eccezioni catturate nel `main`.
- **Commenti:** scopo di ogni classe e metodo, unità di misura, convenzioni scelte (ad esempio il segno di una frazione).
- **Prova finale:** compila con `-Wall` senza avvisi ed esegui il `main`.